

STRUCTURA SI ORGANIZAREA CALCULATOARELOR

Calculatoare 3 2025-2026

CURS 10 Sapt 10 2 decembrie 2025 14:00-16:00

serban@upit.ro

Modul de notare:

Examen:	50%	
Proiect	20%	A3
Laborator:	20%	A2
Test (Midterm):	10%	A1

Obs. Fiecare componenta a notei finale (A1, A2, A3 si Examen) trebuie sa fie promovata cu nota minima 5.

Planificarea activitatilor:

Laborator:

- saptamanal, conform orarului;
- sapt. 13, 14 refaceri laboratoare;

Proiect:

- conform orarului

Curs:

- saptamanal, conform orarului;

Test (Midterm): in saptamana a 10-a (**9 decembrie 2025**)

Continut

Bazele aritmetice ale calculatoarelor electronice

- Sisteme de numeratie

- Reprezentarea numerelor in virgula fixa / mobila

- Operatii aritmetice in virgula fixa / mobila

Proiectarea blocurilor logice pentru operatiile aritmetice in calculatoarele electronice

- Adunarea; scaderea

- Inmultirea

- Impartirea

Proiectarea procesoarelor

- Procesoare RISC

- Setul de instructiuni

- Calea de date (Data Path)

- Calea de control (Control Path)

Bibliografie

David A. Patterson, John L. Hennessy - Computer Organization and Design The Hardware Software Interface [RISC-V Edition] (6th edition Morgan Kaufmann, 2018)

David Harris, Sarah Harris - Digital Design and Computer Architecture (2007)

J. Hennessy, D. Patterson - Computer Architecture. A Quantitative Approach (6th edition Morgan Kaufmann 2019)

William Stallings - Computer Organization and Architecture Designing for Performance (10th edition Pearson 2016)

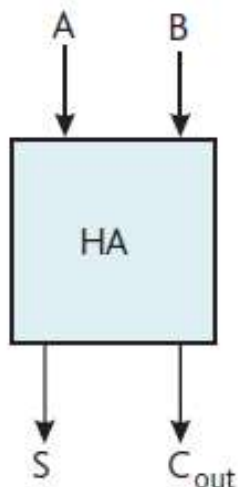
A. Tanenbaum, T. Austin - Structured Computer Organization, (6th edition Pearson 2013)

Karbo M.B. - A complete illustrated guide to the PC hardware (2005)

CIRCUITE PENTRU EFECTUAREA OPERATIILOR ARITMETICE IN PROCESOARE

Circuite pentru efectuarea operatiei de adunare

Semisumatorul – Half Adder



$$C_{OUT} = A \cdot B = \overline{\overline{A \cdot B}} = \overline{\overline{A} \cdot \overline{B}} = \overline{\overline{A} + B}$$

$$S = \overline{A \cdot B} + A \cdot B = A \oplus B$$

$$S = \overline{\overline{A \cdot B} + A \cdot B} = \overline{(\overline{A \cdot B}) \cdot (A \cdot B)}$$

$$S = \overline{(\overline{A \cdot B}) \cdot (A \cdot B)} = \overline{\overline{A} + \overline{B}} + \overline{A + B}$$

Circuit Logic Combinational (CLC)

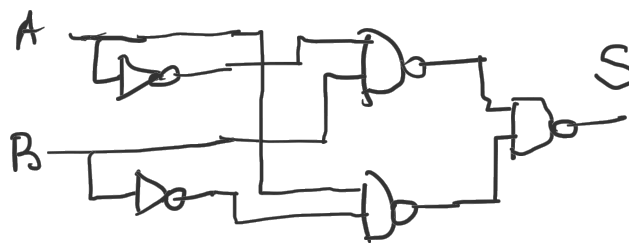
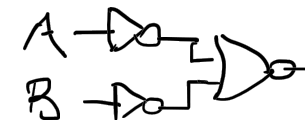
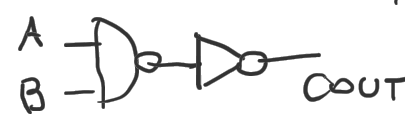
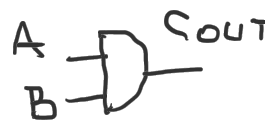
A	B	Cout	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

A, B - operanzi pe 1 bit

Cout - transport (carry)

S - suma

Cout, S - 1 bit

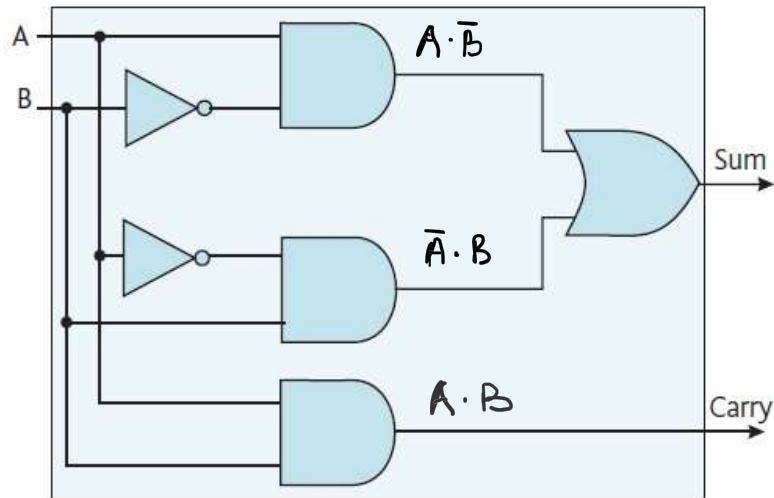


De Morgan

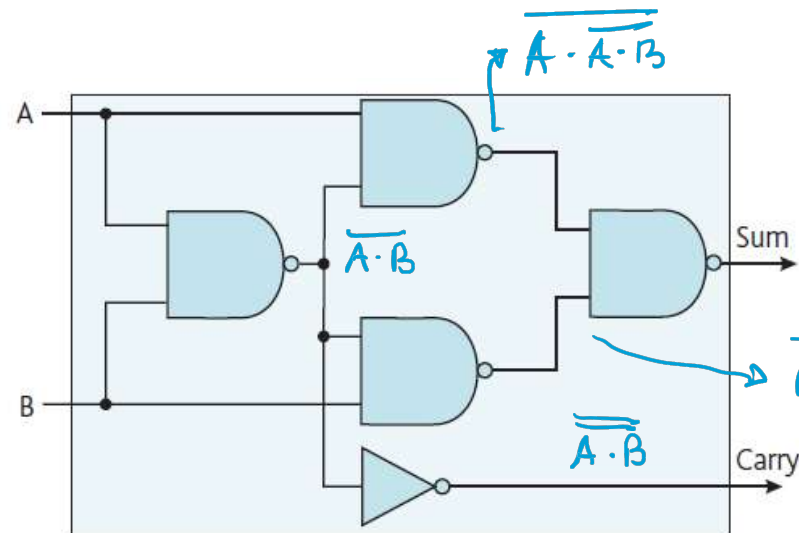
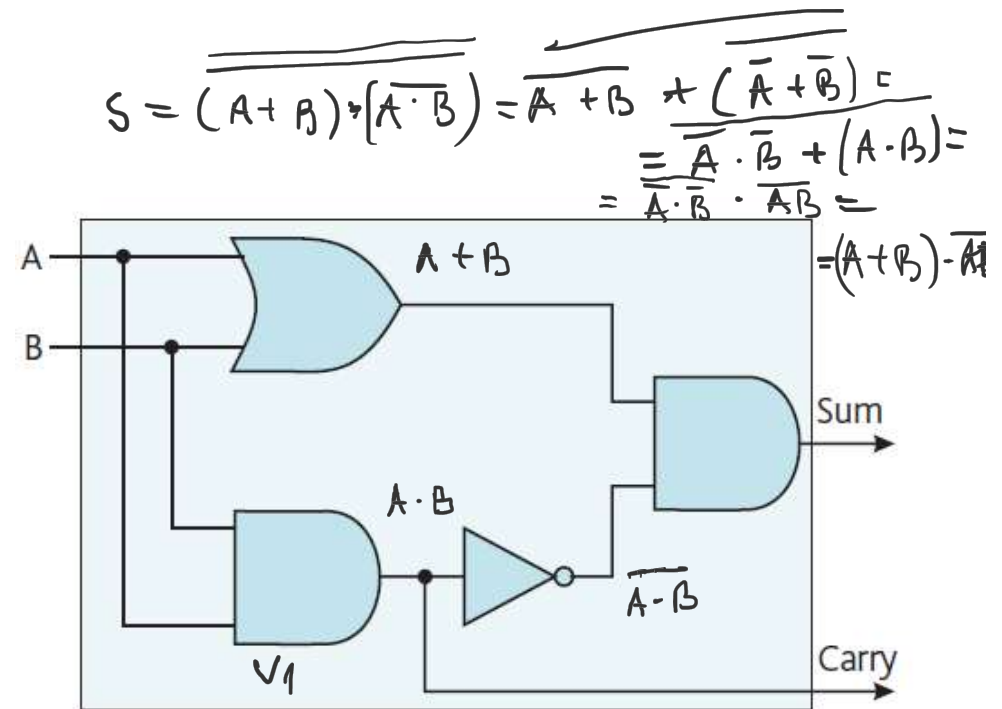
$$\begin{cases} \overline{x \cdot y} = \overline{x} + \overline{y} \\ \overline{x + y} = \overline{x} \cdot \overline{y} \end{cases}$$

$$\begin{cases} \cdot \rightarrow \text{SI} \\ + \rightarrow \text{SAU} \end{cases}$$

Solutii de implementare Semisumator



V1.



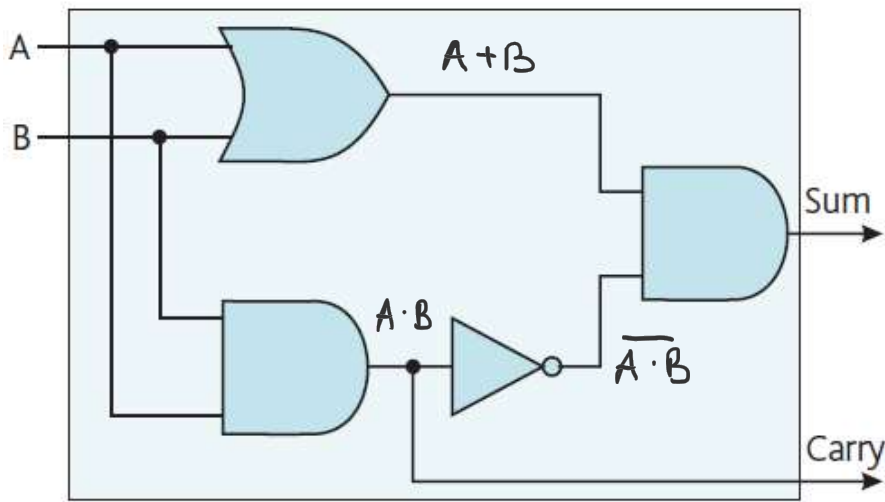
V3

$$\overline{A \cdot \overline{A} B} = \overline{A} + AB$$

$$= \overline{A} \cdot B + A \cdot \overline{B} = A \oplus B$$

$$S = (A+B) \cdot \overline{A \cdot B} = \overline{A+B} + \overline{A \cdot B} = \overline{A} \cdot \overline{B} + (A+B) = \overline{A} \cdot \overline{B} + A \cdot \overline{B} + A \cdot B = (\overline{A} + A) \cdot \overline{B} + A \cdot B = \overline{B} + A \cdot B = (\overline{B} + A) \cdot (\overline{B} + B) = (\overline{B} + A) \cdot 1 = \overline{B} + A = A + \overline{B}$$

$$\begin{aligned} & \overline{A \cdot \overline{A} B \cdot B \cdot \overline{A} B} = \\ & = \overline{[\overline{A} + A \cdot B] \cdot [B + AB]} = \\ & = \overline{\overline{A} + AB} + \overline{B + AB} = \\ & = A \cdot \overline{AB} + B \cdot \overline{AB} = \\ & = \overline{A} \cdot B + A \cdot \overline{B} = \\ & = (\overline{A} + B) \cdot (A + \overline{B}) = \\ & = \overline{A} \cdot A + \overline{A} \cdot \overline{B} + A \cdot B + B \cdot \overline{B} = \\ & = \overline{A} \cdot \overline{B} + A \cdot B = A \oplus B \end{aligned}$$



$$S = (A+B) \cdot \overline{A \cdot B} = \overline{\overline{(A+B)} \cdot \overline{A \cdot B}} = \overline{\overline{A+B} + A \cdot B} =$$

$\uparrow$ double negative $\uparrow$ de Morgan
 $(\overline{x \cdot y} = \overline{x} + \overline{y})$

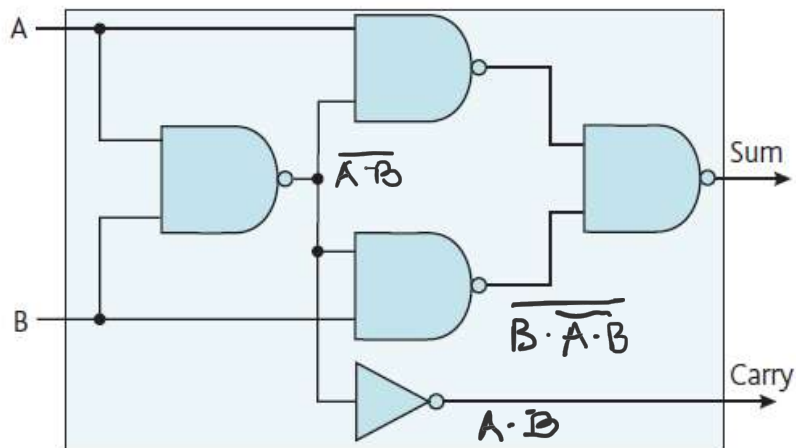
$$= \overline{\overline{A \cdot B} + A \cdot B} = \overline{\overline{A \cdot B}} \cdot \overline{A \cdot B} =$$

$\uparrow$ de Morgan $\uparrow$ de Morgan

$$= (A+B) \cdot \overline{A \cdot B} = (A+B) \cdot (\overline{A} + \overline{B}) =$$

$\uparrow$ de Morgan $\uparrow$ de Morgan

$$= \underbrace{A \cdot \overline{A}}_0 + A \cdot \overline{B} + \overline{A} \cdot B + \underbrace{B \cdot \overline{B}}_0 = A \cdot \overline{B} + \overline{A} \cdot B = A \oplus B$$



$$S = \overline{A \cdot \overline{A \cdot B}} \cdot \overline{B \cdot \overline{A \cdot B}} = \overline{\overline{A} + A \cdot B} \cdot \overline{\overline{B} + A \cdot B} =$$

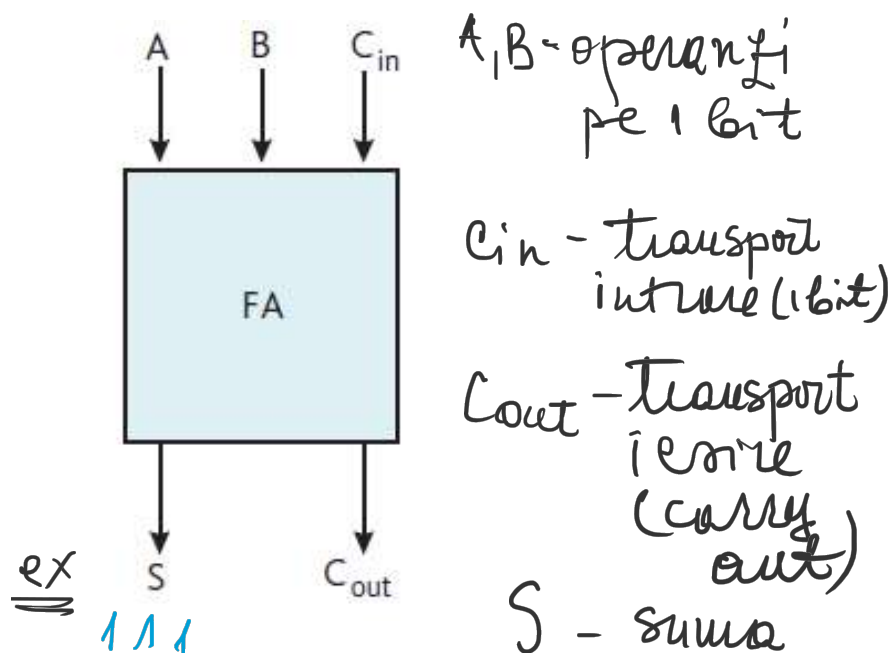
$\uparrow$ DM $\uparrow$ DM

$$= \overline{\overline{A} + A \cdot B} \cdot \overline{\overline{B} + A \cdot B} = A \cdot \overline{A \cdot B} + B \cdot \overline{A \cdot B} =$$

$\uparrow$ DM $\uparrow$ DM

$$= A \cdot (\overline{A} + \overline{B}) + B(\overline{A} + \overline{B}) = \overline{A}A + A\overline{B} + \overline{A}B + \overline{B}B = A\overline{B} + \overline{A}B = A \oplus B$$

Sumatorul complet elementar- Full Adder



A	B	C _{in}	S (sum)	C _{out} (carry-out)
0	0	0	0	0
0	1	0	1 *	0
1	0	0	1 *	0
1	1	0	0	1 *
0	0	1	1 *	0
0	1	1	0	1 *
1	0	1	0	1 *
1	1	1	1 *	1 *

ex

```

  111
1011 *
1101
-----
11000
  
```

Half Adder

Full Adder

adunare

$$\begin{cases}
 S = \bar{A} \cdot \bar{B} \cdot \bar{C}_{in} + \bar{A} \cdot B \cdot \bar{C}_{in} + A \cdot \bar{B} \cdot \bar{C}_{in} + A \cdot B \cdot \bar{C}_{in} \\
 C_{out} = A \cdot B \cdot \bar{C}_{in} + \bar{A} \cdot B \cdot C_{in} + A \cdot \bar{B} \cdot C_{in} + A \cdot B \cdot C_{in}
 \end{cases}$$

$X = \underbrace{X}_{SAU} + \underbrace{X}_{SAU} = \underbrace{X}_{SAU} + \underbrace{X}_{SAU} + \underbrace{X}_{SAU}$

$$C_{out} = \underbrace{A \cdot B \cdot \bar{C}_{in} + A \cdot B \cdot C_{in}}_{A \cdot B} + \underbrace{\bar{A} \cdot B \cdot C_{in} + A \cdot \bar{B} \cdot C_{in}}_{B \cdot C_{in}} + \underbrace{A \cdot \bar{B} \cdot \bar{C}_{in} + A \cdot B \cdot \bar{C}_{in}}_{A \cdot (\bar{B} + B)}$$

$$C_{out} = A \cdot B \cdot (\bar{C}_{in} + C_{in}) + B \cdot C_{in} (\bar{A} + A) + A \cdot C_{in} (\bar{B} + B) \quad (y + \bar{y} = 1)$$

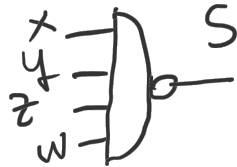
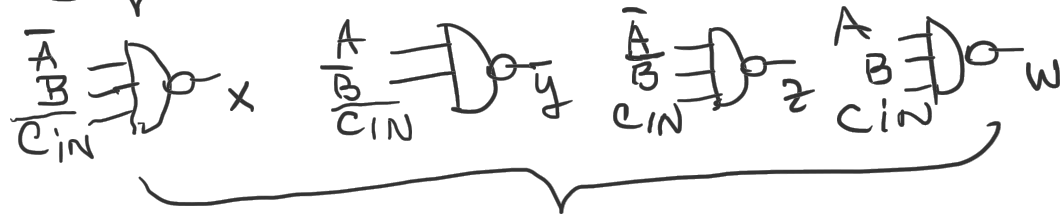
$$C_{out} = A \cdot B + B \cdot C_{in} + A \cdot C_{in}$$

$$S = \overline{A} \cdot \overline{B} \cdot \overline{C_{IN}} + \overline{A} \cdot \overline{B} \cdot C_{IN} + \overline{A} \cdot B \cdot \overline{C_{IN}} + A \cdot B \cdot C_{IN}$$

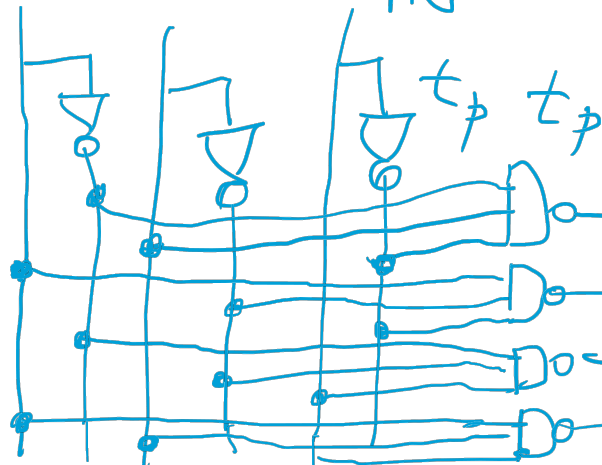
$$C_{OUT} = \overline{A \cdot B + B \cdot C_{IN} + A \cdot C_{IN}} \quad \text{DM} \quad \overline{AB \cdot B \cdot C_{IN} \cdot A \cdot C_{IN}}$$

$$S = \overline{A} \cdot \overline{B} \cdot C_{in} + A \cdot \overline{B} \cdot C_{in} + \overline{A} \cdot B \cdot C_{in} + A \cdot B \cdot C_{in} = \underline{\underline{DM}}$$

$$= \underbrace{\bar{A} \cdot B \cdot C_{in}}_{\text{1}} \cdot \underbrace{A \cdot \bar{B} \cdot C_{in}}_{\text{1}} \cdot \underbrace{\bar{A} \cdot \bar{B} \cdot C_{in}}_{\text{1}} \cdot \underbrace{A \cdot B \cdot C_{in}}_{\text{1}}$$

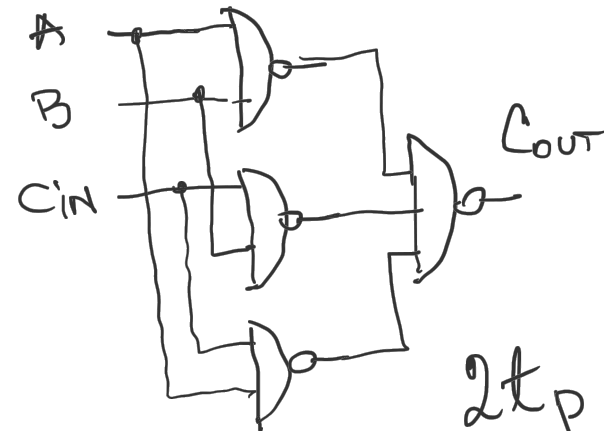


A B C in

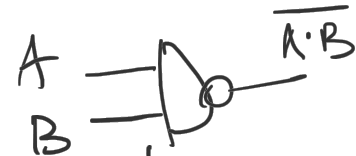


$$t_p \Rightarrow 3t_p$$

Sports



2tp



4 part



$$t_p = t_1 - t_0$$

(Give propagation)

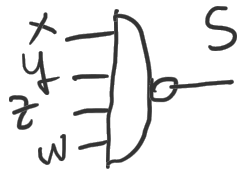
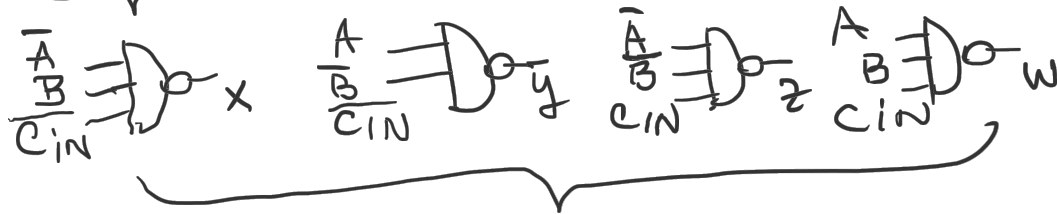
DM - Relativ

De Morgan

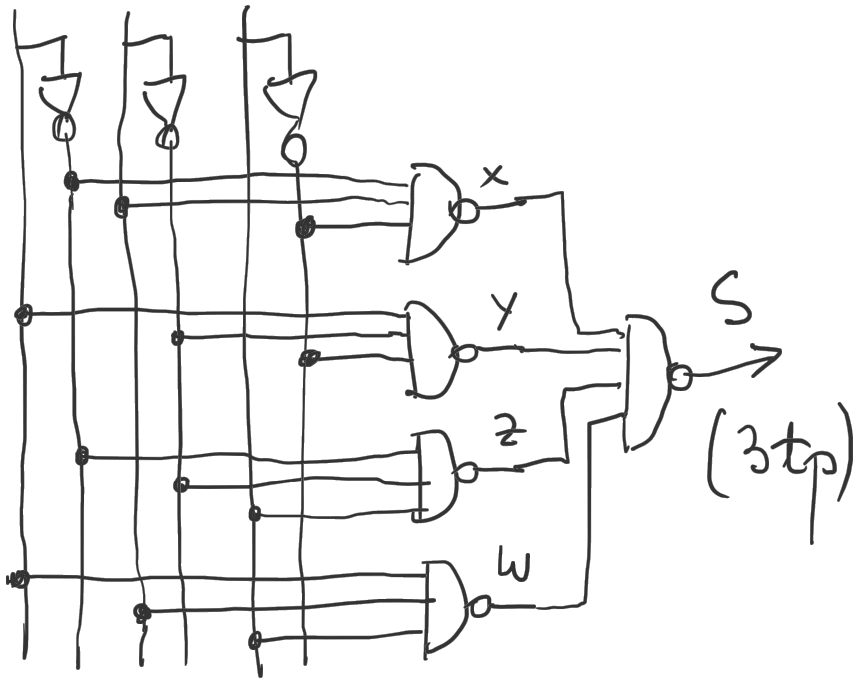
$$\overline{x+y} = \overline{x} \cdot \overline{y}$$

$$S = \overline{A} \cdot B \cdot \overline{C_{IN}} + A \cdot \overline{B} \cdot \overline{C_{IN}} + \overline{A} \cdot \overline{B} \cdot C_{IN} + A \cdot B \cdot C_{IN} = \text{DM}$$

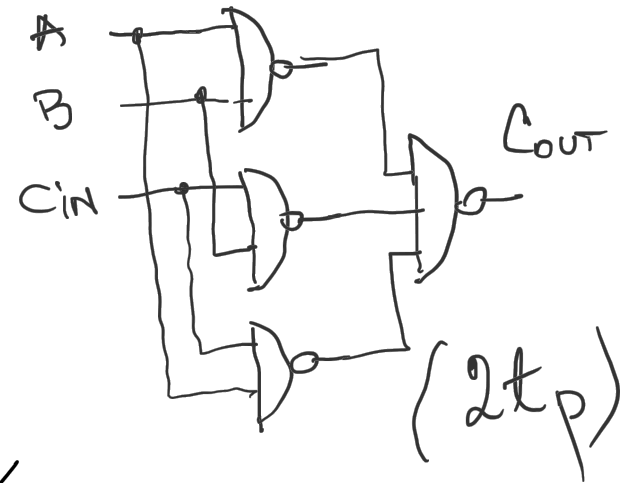
$$= \underbrace{\overline{A} \cdot B \cdot \overline{C_{IN}}}_{x} \cdot \underbrace{A \cdot \overline{B} \cdot \overline{C_{IN}}}_{y} \cdot \underbrace{\overline{A} \cdot \overline{B} \cdot C_{IN}}_{z} \cdot \underbrace{A \cdot B \cdot C_{IN}}_{w}$$



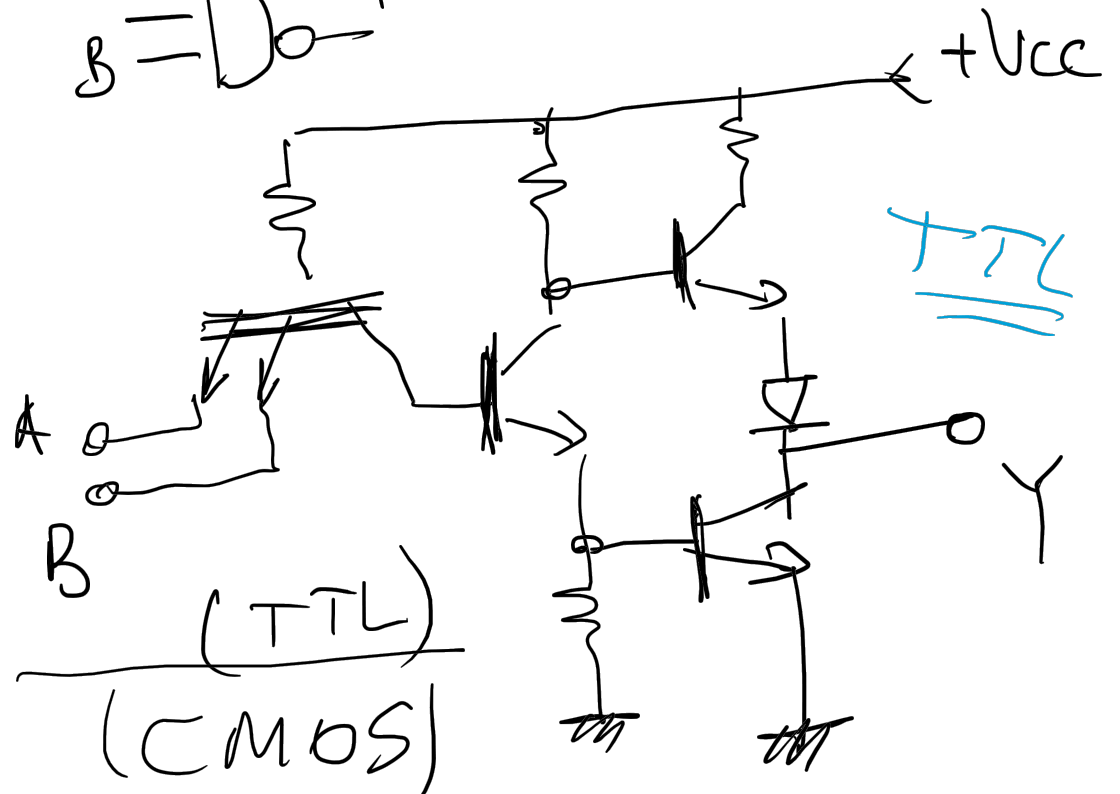
A B C_{IN}



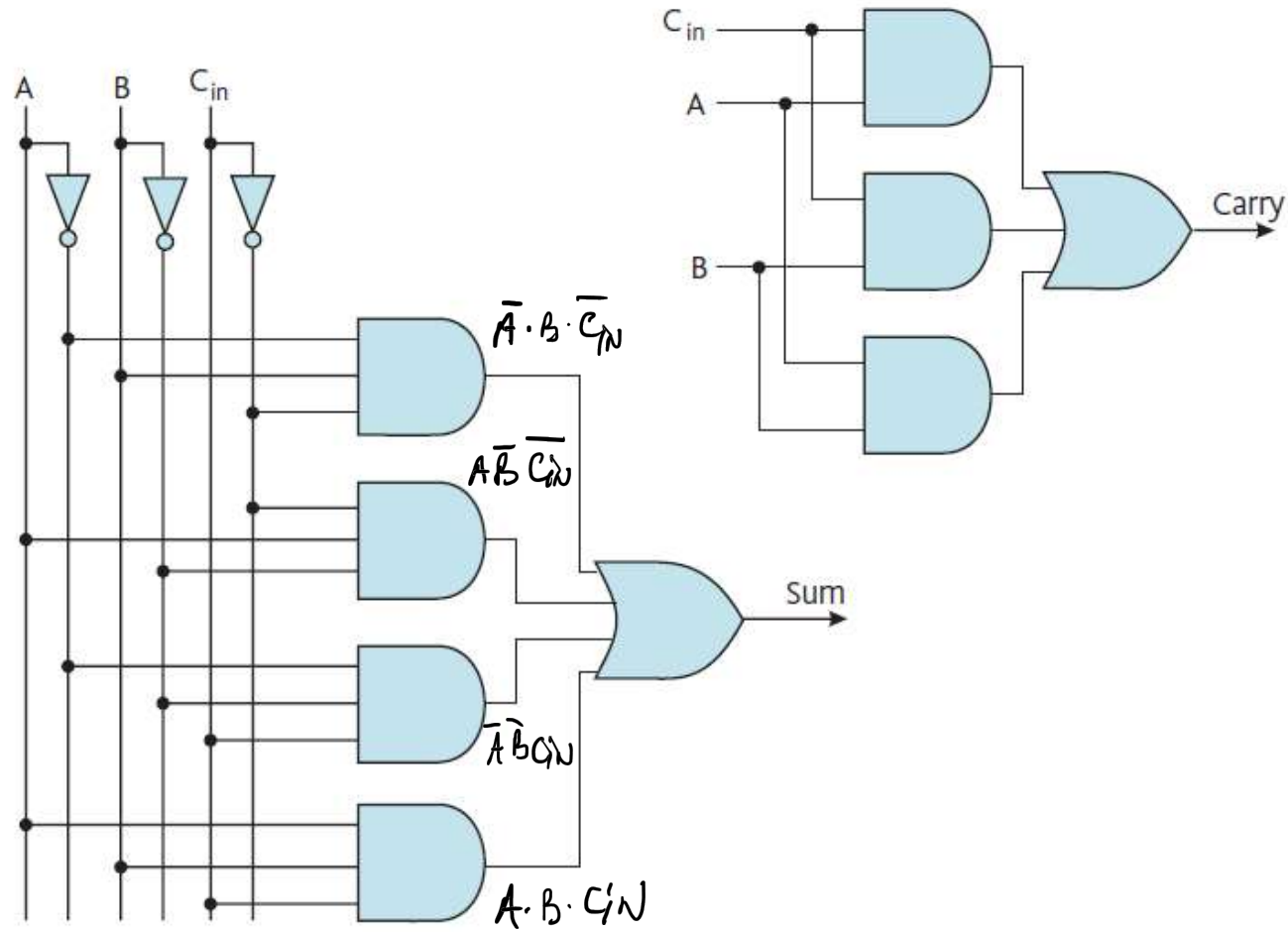
$$C_{OUT} = \overline{A} \cdot B \cdot \overline{B} \cdot C_{IN} \cdot \overline{A} \cdot C_{IN}$$



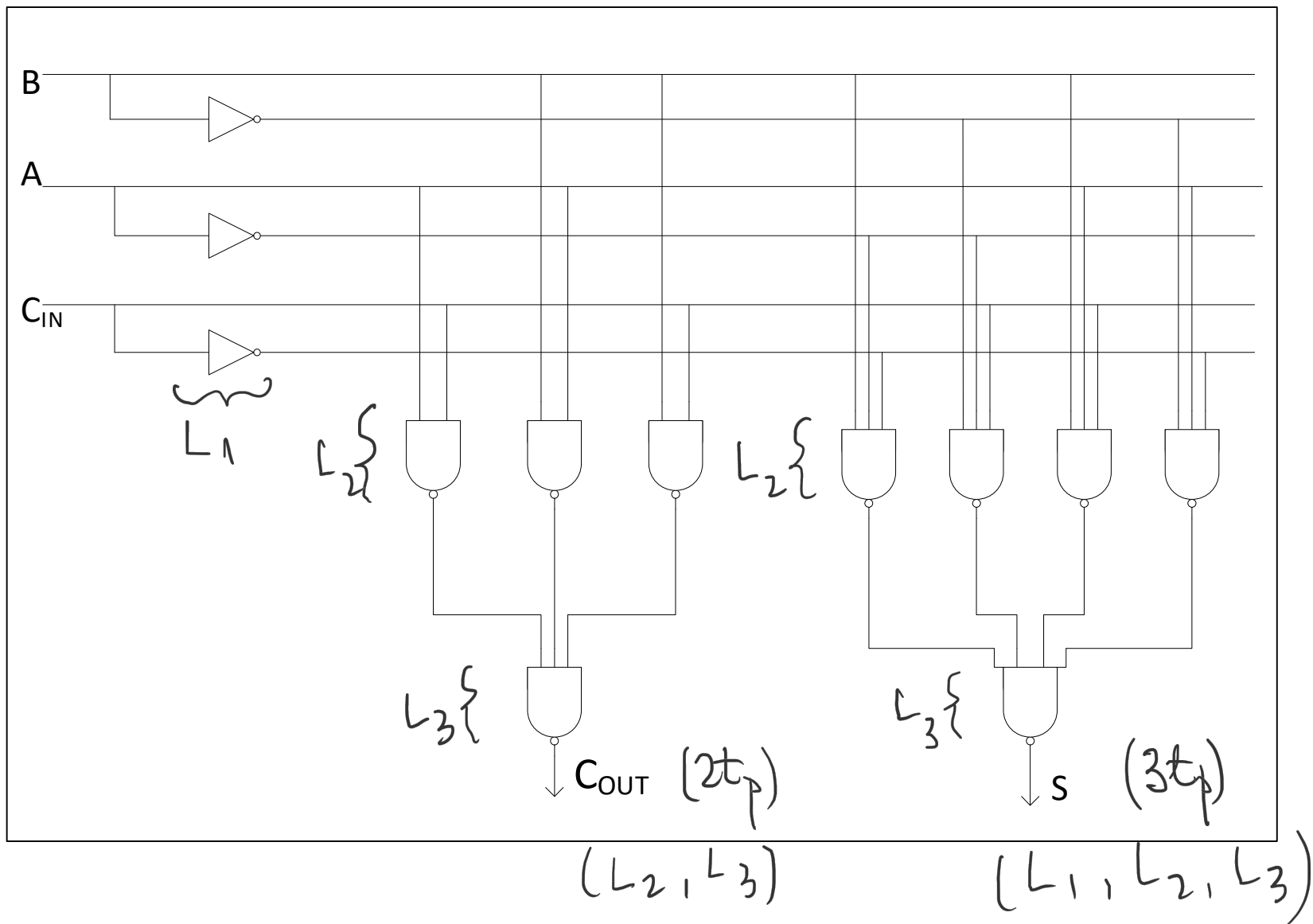
$$A \cdot B = Y$$



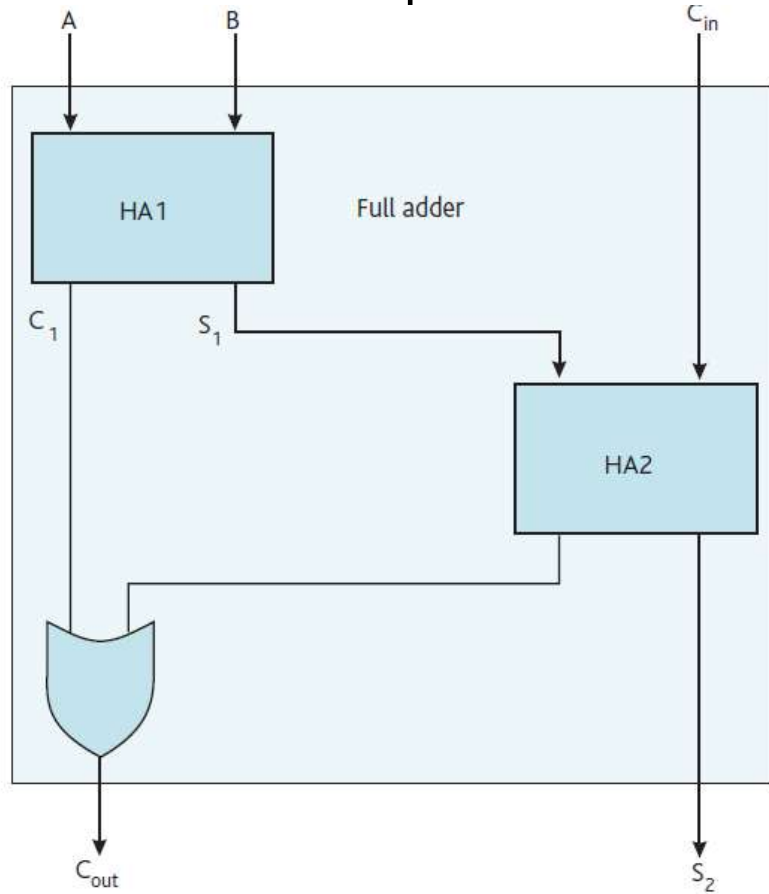
Solutie de implementare Sumator Complet



Solutie de implementare Sumator Complet



Solutie de implementare Sumator Complet folosind Semisumator

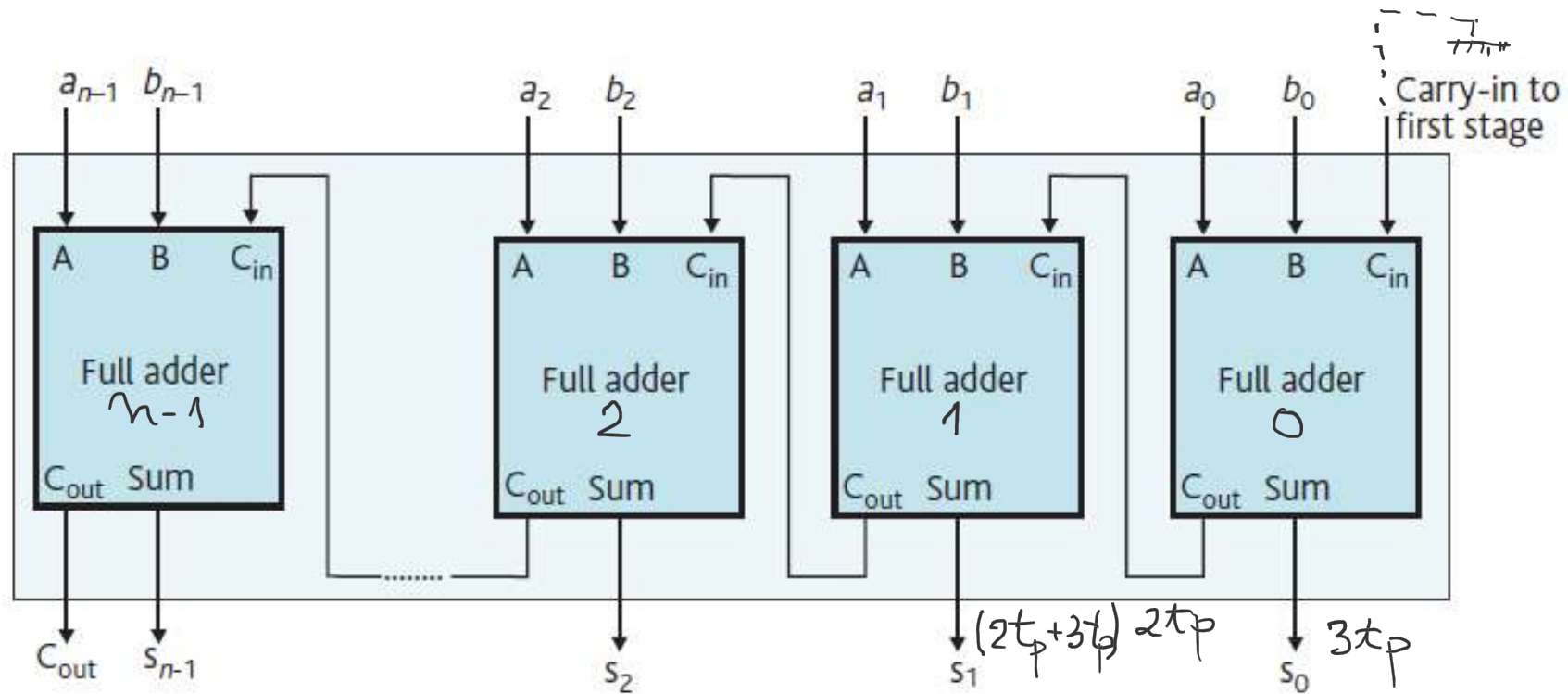


C_{in}	A	B	S_1	C_1	S_2	C_2	C_{out}
0	0	0	0	0	0	0	0
0	0	1	1	0	1	0	0
0	1	0	1	0	1	0	0
0	1	1	0	1	0	0	1
1	0	0	0	0	1	0	0
1	0	1	1	0	0	1	1
1	1	0	1	0	0	1	1
1	1	1	0	1	1	0	1

Handwritten annotations below the table:

- Under the first three columns (A, B, S_1), a bracket is drawn with the handwritten text "Semisumator 1".
- Under the next three columns (C_1 , S_2 , C_2), a bracket is drawn with the handwritten text "Semisumator 2".
- Below "Semisumator 1", the number "1" is written, with an arrow pointing to a circled S_1 .
- Below "Semisumator 2", the number "2" is written, with an arrow pointing to a circled C_{out} .

Sumator Paralel – Ripple-Carry Adder (sumator cu transport propagat)



$A(a_{n-1} a_{n-2} \dots a_2 a_1 a_0)$ – n biti

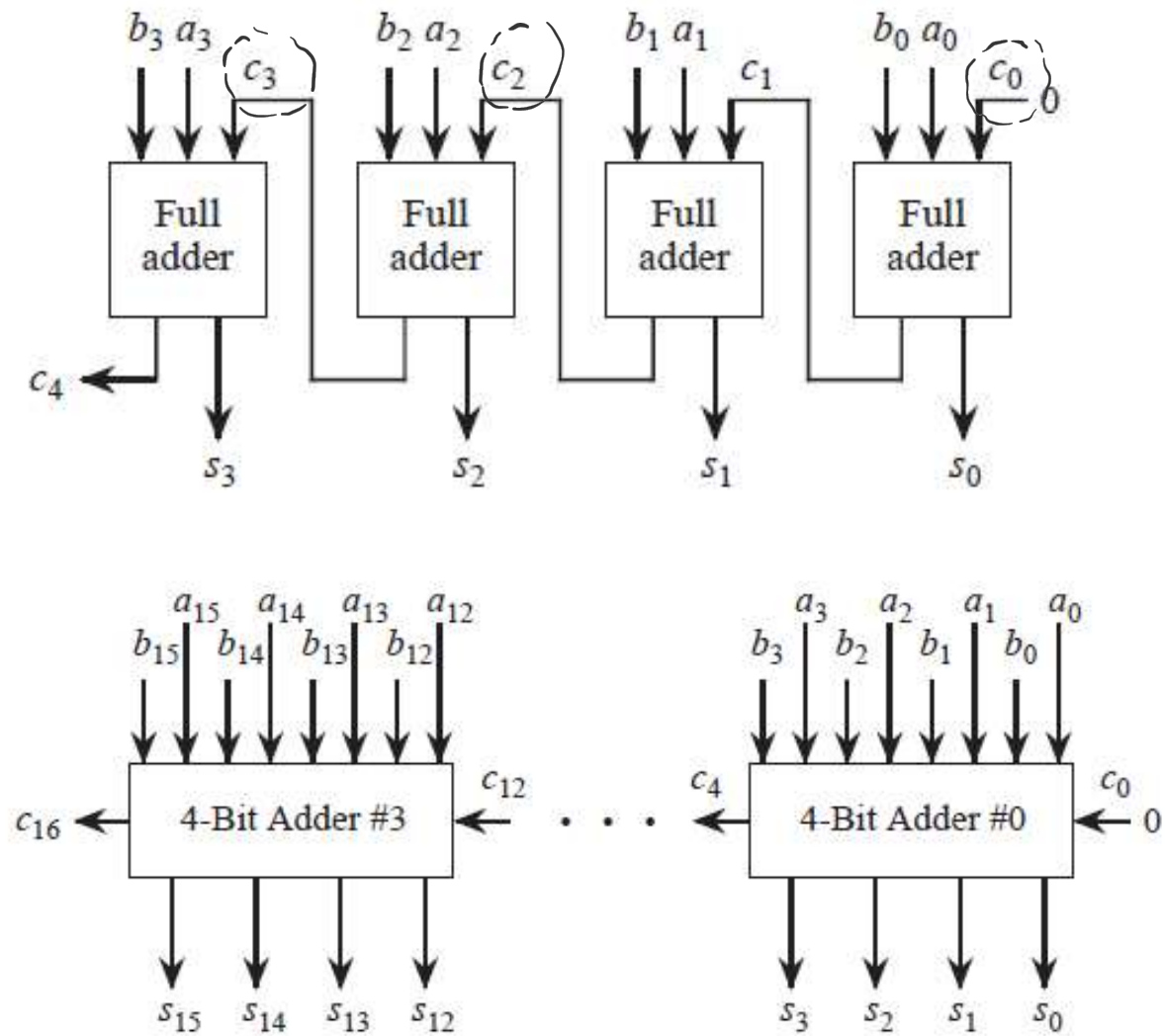
$B(b_{n-1} b_{n-2} \dots b_2 b_1 b_0)$ – n biti

$$\begin{cases} S_1 \rightarrow 2t_p + 3t_p = 5t_p \\ C_1 \rightarrow 2t_p + 2t_p = 4t_p \\ S_2 \rightarrow 4t_p + 3t_p = 7t_p \\ C_2 \rightarrow 4t_p + 2t_p = 6t_p \\ S_3 \rightarrow 6t_p + 3t_p = 9t_p \\ C_3 \rightarrow 6t_p + 2t_p = 8t_p \end{cases}$$

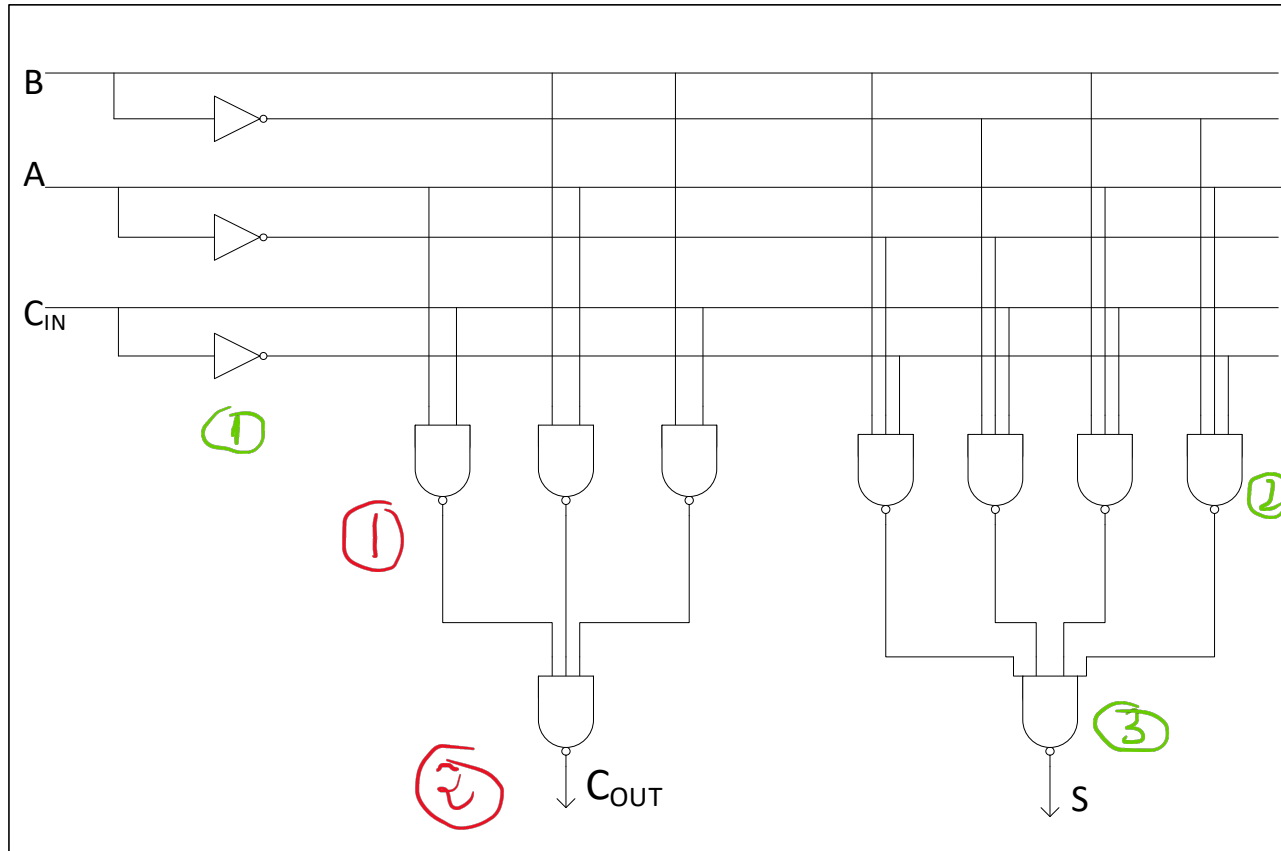
$$\begin{cases} C_n \rightarrow 2(n+1) \cdot t_p \\ S_n \rightarrow (2n+3) \cdot t_p \end{cases}$$

Portii logice $\rightarrow 12 \cdot n$

Sumator Paralel cu Transport Propagat – Ripple-Carry Adder



Solutie de implementare Sumator Complet



$$C \rightarrow 2t_p$$

$$S \rightarrow 3t_p$$

$$c_{i+1} = b_i c_i + a_i c_i + a_i b_i$$

$$= \overline{b_i c_i} \cdot \overline{a_i c_i} \cdot \overline{a_i b_i}$$

↑ De Morgan

(C_{OUT}) $c_{i+1} = b_i c_i + a_i c_i + a_i b_i$

$$c_{i+1} = a_i b_i + (a_i + b_i) c_i$$

$$G_i = a_i b_i \quad P_i = a_i + b_i$$

$$c_{i+1} = G_i + P_i c_i$$

$$c_{i+1} = (b_i \wedge c_i) \vee (a_i \wedge c_i) \vee (a_i \wedge b_i)$$

$$s_i = \overline{a_i} \overline{b_i} c_i + \overline{a_i} b_i \overline{c_i} + a_i \overline{b_i} \overline{c_i} + a_i b_i c_i$$

G - Generate

P - propagate

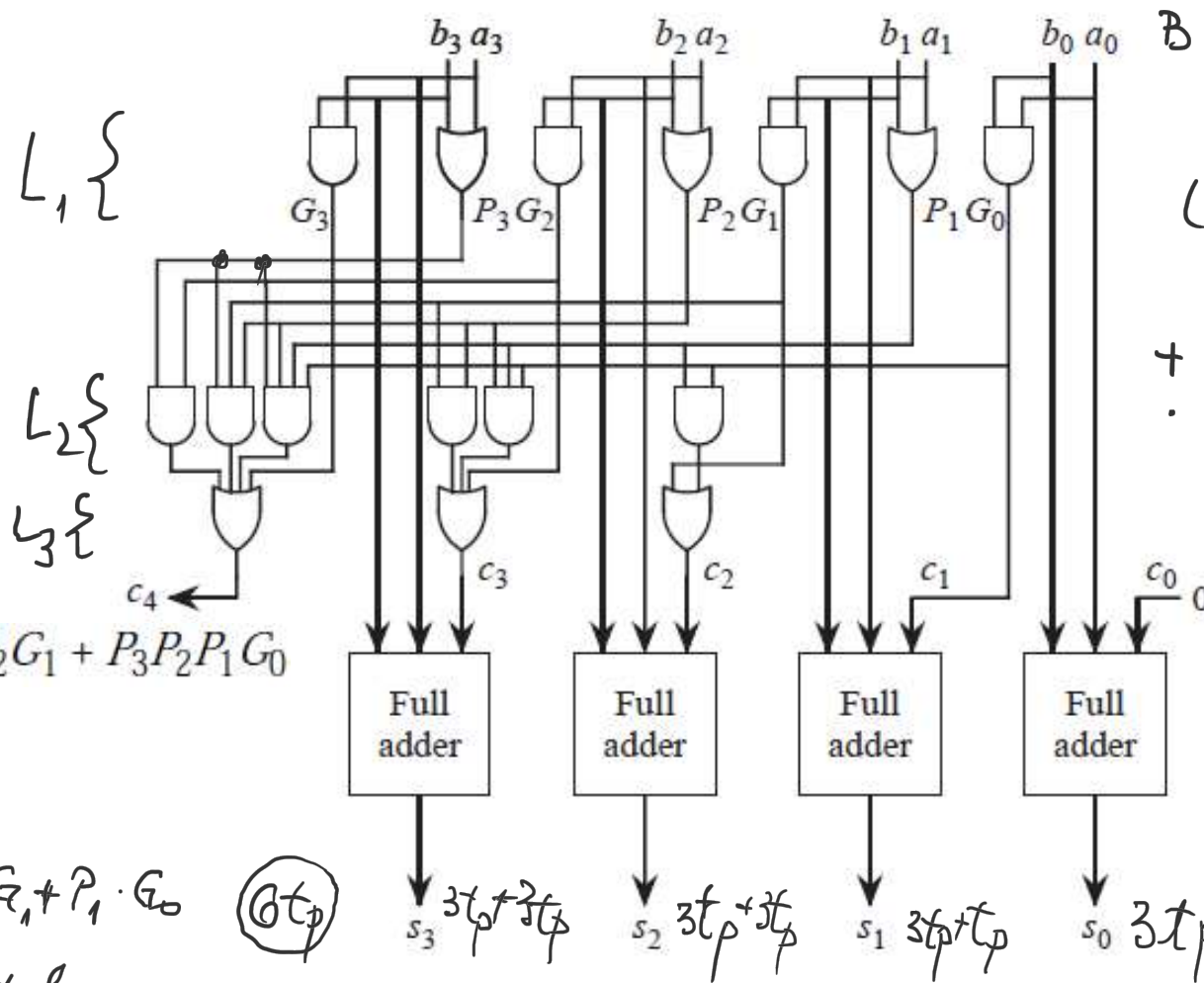
• - Si logic (1)

+ - SAU logic (V)

Sumator cu Transport Anticipat – Carry Look-Ahead Adder

$$\textcircled{*} \quad c_{i+1} = G_i + P_i c_i$$

**CARRY
LOOK
AHEAD
ADDER**



$A (a_3 a_2 a_1 a_0)$
 $B (b_3 b_2 b_1 b_0)$
 $A + B$
 (suma aritmetică)
 + SAU Logic
 · Si Logic

**END 11
nov
2025**

$$c_0 = 0$$

$$c_1 = G_0 = a_0 \cdot b_0$$

$$\textcircled{*} c_2 = G_1 + P_1 \cdot c_1 = G_1 + P_1 \cdot G_0 \quad (6t_p)$$

$$G_1 = a_1 \cdot b_1 \quad P_1 = a_1 + b_1$$

$$\textcircled{*} c_3 = G_2 + P_2 \cdot c_2 = G_2 + P_2 \cdot (G_1 + P_1 \cdot G_0) = G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0$$

$$c_4 = G_3 + P_3 \cdot c_3 = G_3 + P_3 \cdot (G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0) = G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0$$

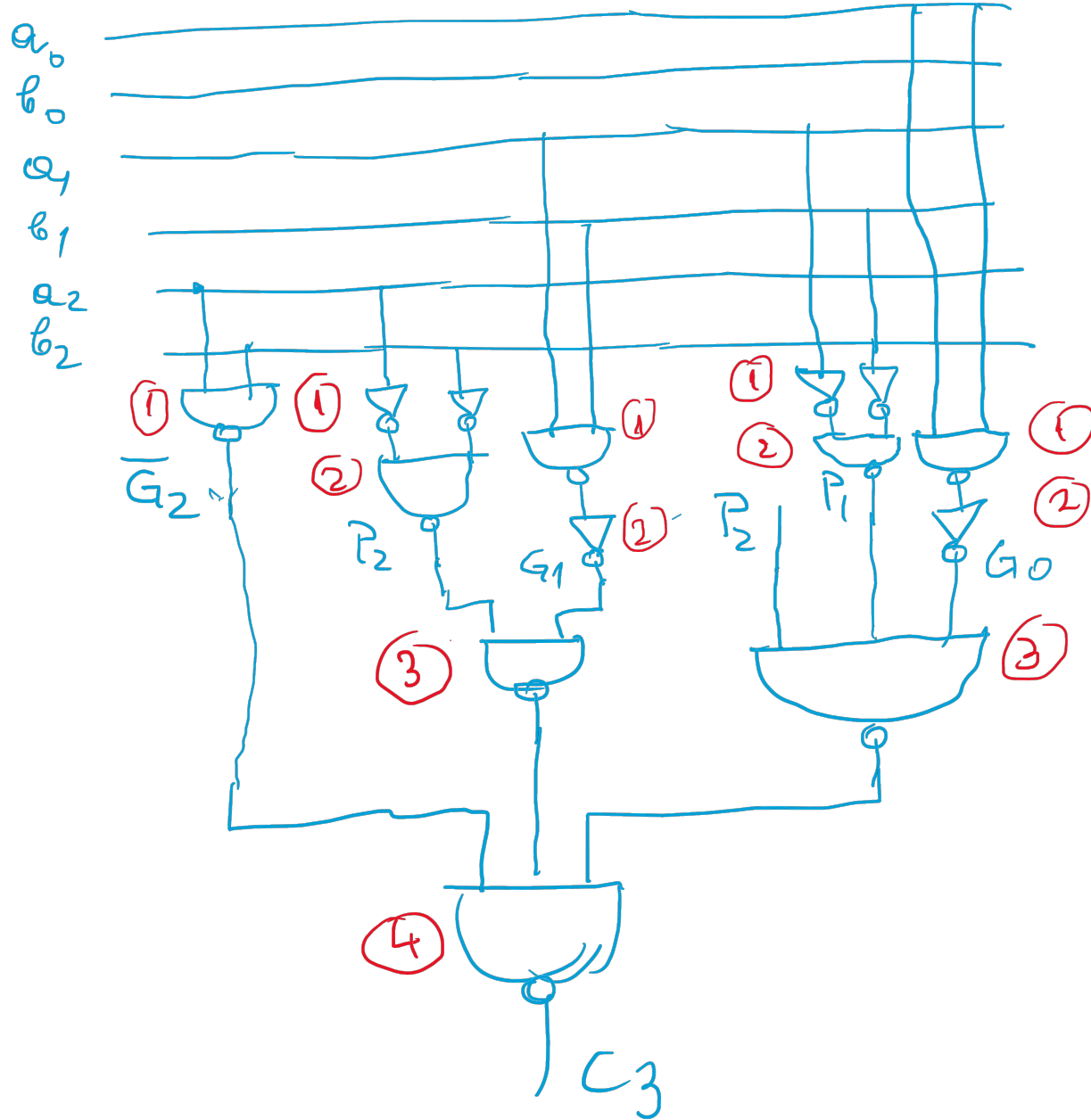
$$G_2 = a_2 \cdot b_2 \quad P_2 = a_2 + b_2$$

$$G_3 = a_3 \cdot b_3 \quad P_3 = a_3 + b_3$$

$$c_3 = \overline{G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0} = \overline{G_2} \cdot \overline{P_2 \cdot G_1} \cdot \overline{P_2 \cdot P_1 \cdot G_0}$$

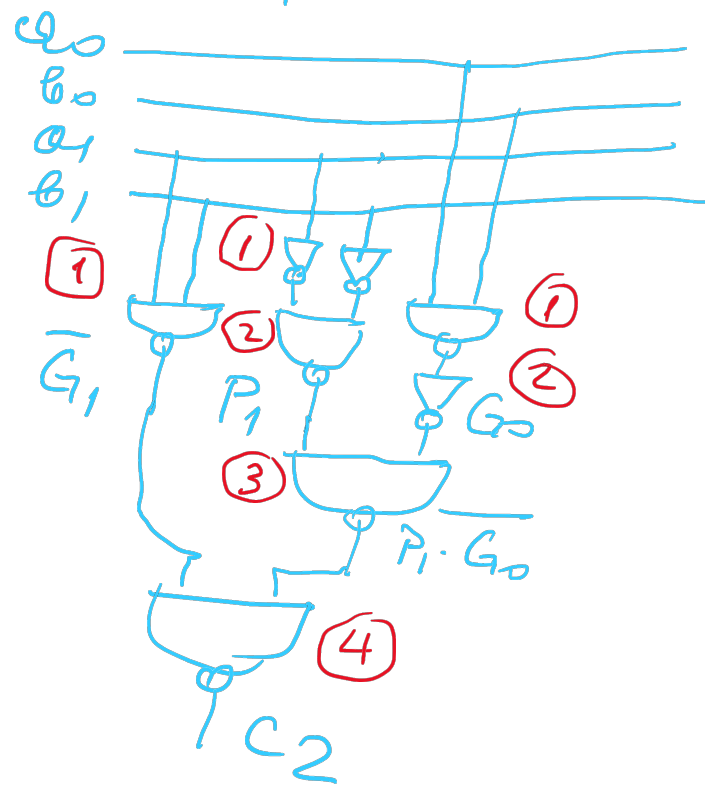
$$G_i = a_i \cdot b_i$$

$$P_i = \overline{a_i + b_i} = \overline{a_i} \cdot \overline{b_i}$$

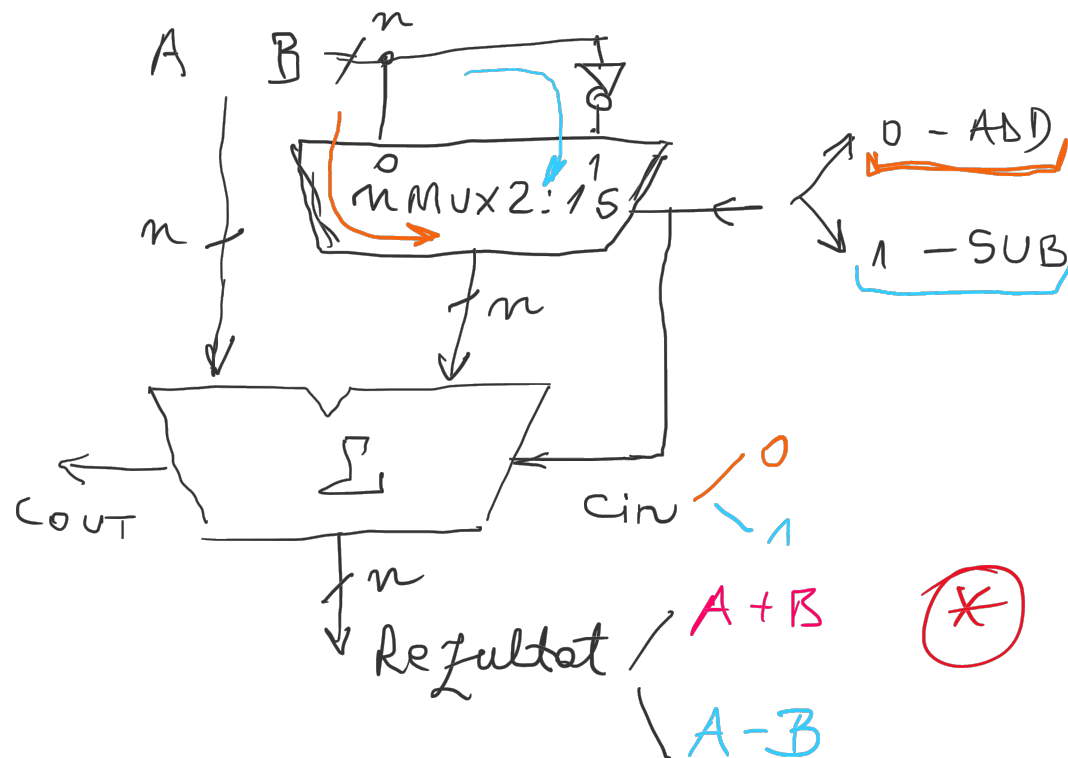
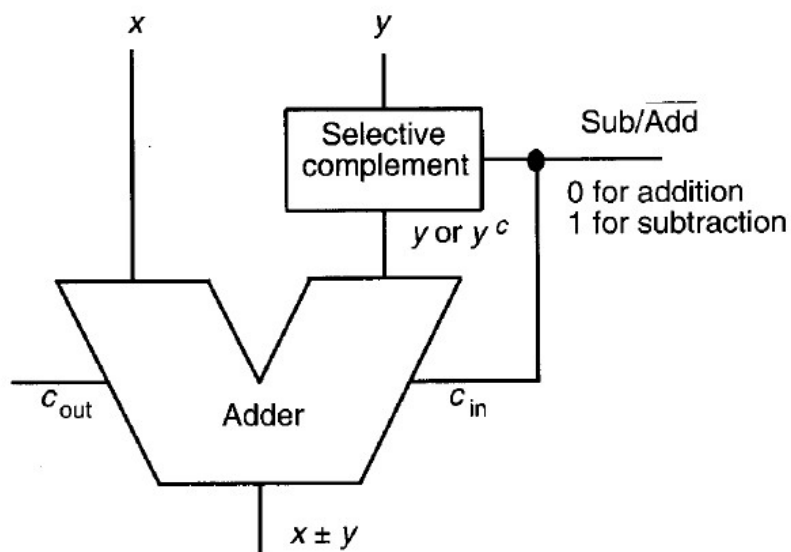


$$C_{out} \rightarrow 4t_p$$

$$C_2 = \overline{G_1 + P_1 \cdot G_0} = \overline{G_1} \cdot \overline{P_1 \cdot G_0}$$



Circuit pentru efectuarea operatiei de scadere



$$x - y = x + (-y) \quad \text{CC2}$$

$$-y = \overline{y} + 1 \quad \text{CC2}$$

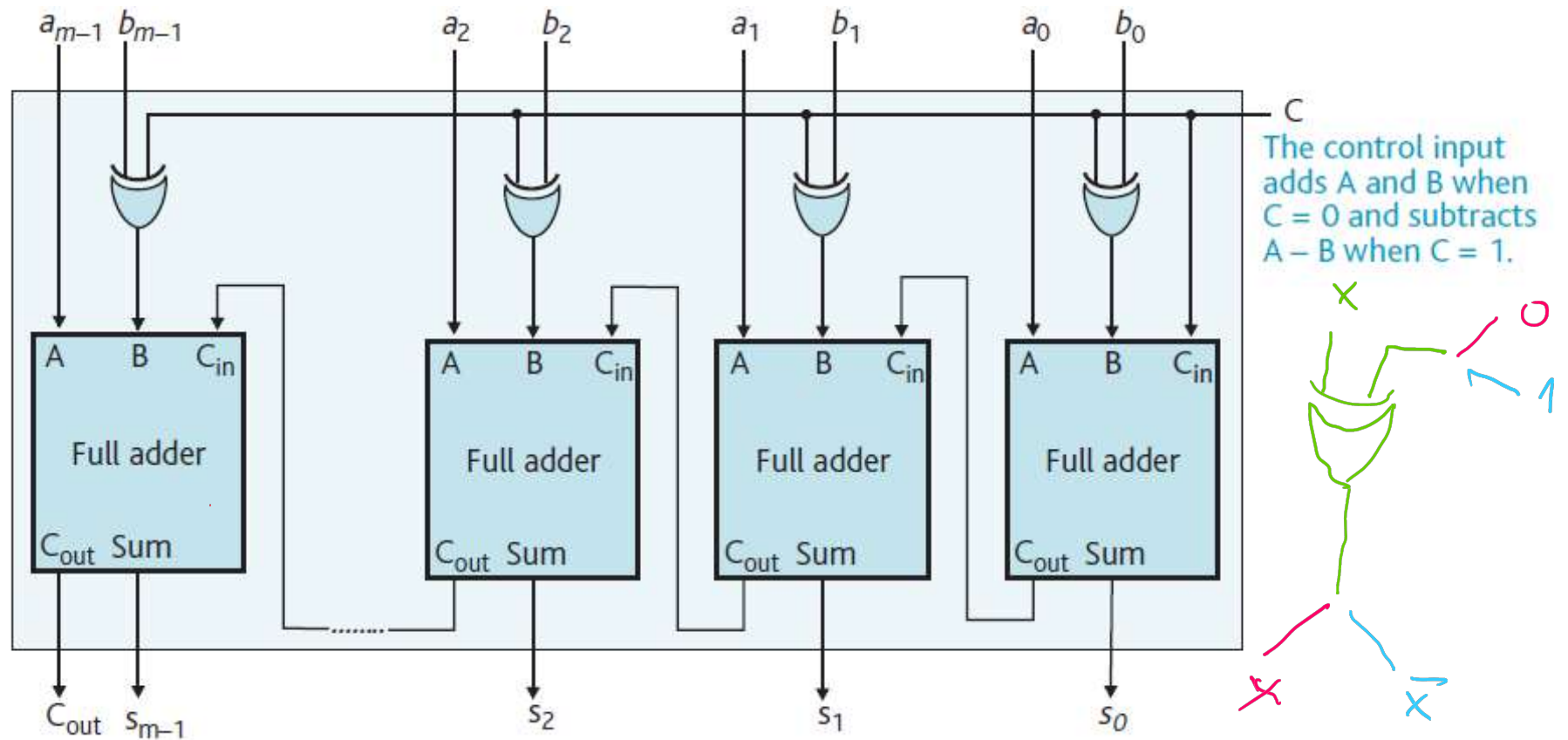
~~(X)~~ - adunare !!

Σ -sumator pe n biti (ripple carry)
 $\Sigma \rightarrow A + B + 0 = A + B$ ~~(X)~~ (carry look ahead)

$$\Sigma \rightarrow A + \overline{B} + 1 = A + (-B) = A - B$$

+ , - notati
 ordon, scad
 + adunare

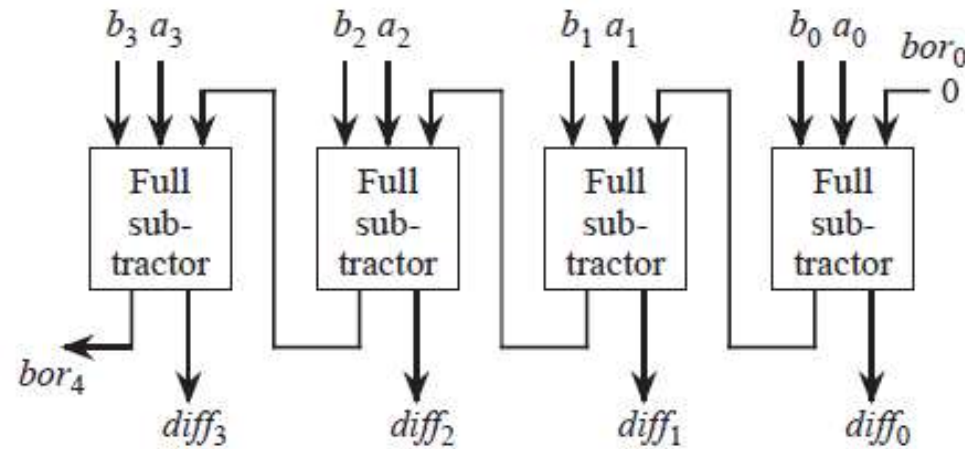
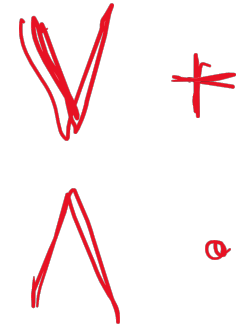
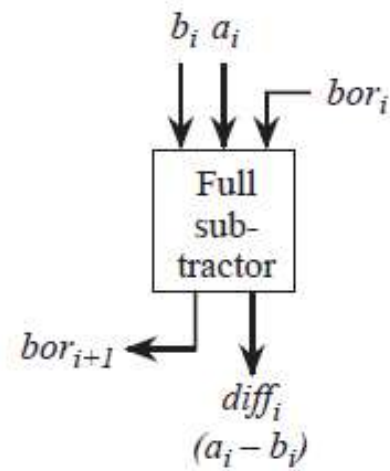
Circuit pentru efectuarea operatiei de scadere



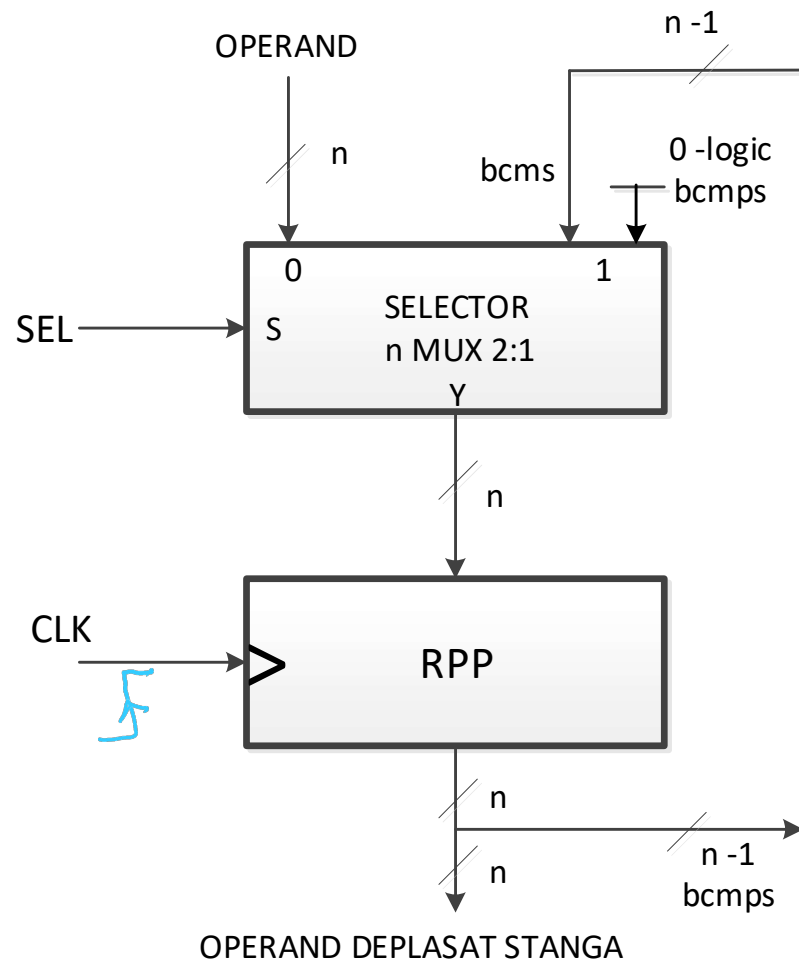
Circuit pentru efectuarea operatiei de scadere

borrow

a_i	b_i	bor_i	$diff_i$	bor_{i+1}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1



Circuit pentru deplasare operand stanga



(*) Deplasarea stanga
este echiv cu $\times 2$!
 $101_{(2)} = 5$
 $\leftarrow 1010_{(2)} = 10$

bcms
 $\begin{array}{r} 101 \times \\ 110 \\ \hline 009 \\ 1010 \\ 10100 \end{array}$
 bcmps

etape de functionare

1) aplic operand

SEL = 0

CLK $\uparrow$

operandul apare la
iesirile RPP

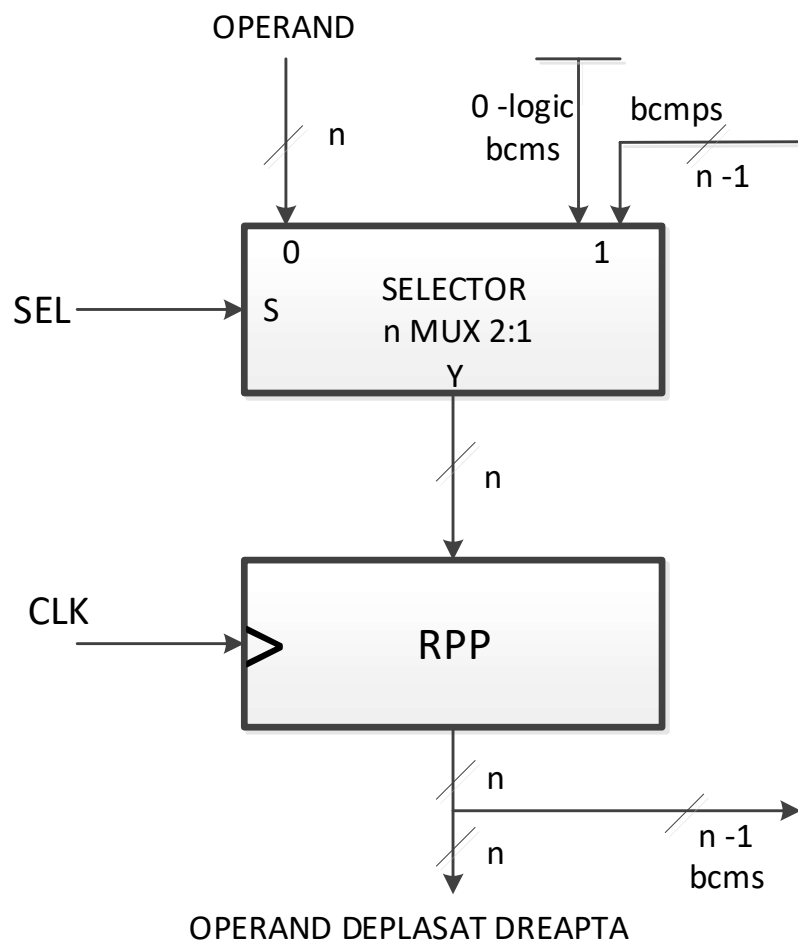
2) SEL = 1

intrare de tip 1 de la
selector se aplica RPP

CLK $\uparrow$

operandul deplasat stanga
cu 0 pozitie apare la
iesirile RPP

Circuit pentru deplasare operand dreapta



$$2^5 \ 2^4 \ 2^3 \ 2^2 \ 2^1 \ 2^0$$

$$A = \underline{110110}$$

dispare

$$B = \underline{011011}$$

$$A = 2^5 + 2^4 + 2^2 + 2^1 = 32 + 16 + 4 + 2 = 54_{(10)}$$

$$B = 2^4 + 2^3 + 2^1 + 2^0 = 16 + 8 + 2 + 1 = 27_{(10)}$$

$$B = A/2$$

- 1) - Se aplică operandul celor n MUX 2:1 pe intrările de tip 0.
 - SEL = 0
 - CLK $\downarrow$
- 2) - SEL = 1
 - CLK $\downarrow$

Inmultirea a 2 operanzi binari

1. Înmulțirea implică generarea unor produse parțiale, câte unul pentru fiecare bit al înmulțitorului. Aceste produse sunt apoi însumate pentru a se obține produsul final.
2. Dacă bitul testat al înmulțitorului este 0, produsul parțial corespunzător este 0. Dacă acest bit este 1, produsul parțial este egal cu deînmulțitul (deplasat stanga cu un numar corespunzator de pozitii, in concordanta cu pozitia bitului din inmultitor)
3. Pentru obținerea produsului final, fiecare produs parțial este deplasat la stânga cu o poziție relativ la produsul parțial precedent.
4. Înmulțirea a două numere binare fără semn de câte n biți are ca rezultat un produs cu o lungime de până la $2n$ biți.

Față de înmulțirea obișnuită, se pot efectua unele modificări pentru ca operația să fie mai eficientă.

* Se poate realiza cumulara produselor parțiale într-un registru, deci adunarea lor pe măsură ce ele sunt obținute, în locul adunării tuturor produselor la final. Deoarece nu este necesară memorarea tuturor produselor parțiale, vor fi necesare mai puține registre. După fiecare produs parțial obținut, deînmulțitul este deplasat la stânga cu o poziție.

* Pentru fiecare bit de 1 din înmulțitor, este necesară o adunare și o deplasare. Pentru fiecare bit de 0 din înmulțitor, produsul parțial este 0, fiind necesară numai o deplasare.

Inmultirea a 2 operanzi binari (cont.)

Rezultă că operația de înmulțire se poate efectua conform următorului algoritm:

1. Se inițializează produsul cu zero.
2. Se testează biții înmulțitorului, începând cu bitul cel mai puțin semnificativ. Dacă bitul testat este 1, se adună deînmulțitul la produs.
3. După fiecare operație, la începutul pasului următor, se deplasează deînmulțitul la stânga cu o poziție (produsul partial se obtine din deinmultit).

Dacă numerele sunt cu semn, semnul rezultatului se poate stabili în funcție de semnul operanzilor: dacă numerele au același semn, produsul este pozitiv, iar în caz contrar produsul este negativ. Pentru operanzi cu $n+1$ biți (un bit pentru semn), produsul va avea $2n+1$ biți.

Produsul se păstrează într-un registru. Acest registru necesită $2n$ poziții, unde n este numărul biților de mărime ai deînmulțitului și înmulțitorului. Deoarece deînmulțitul trebuie deplasat la stânga în fiecare pas al operației, și pentru deînmulțit este necesar un registru de lungime dublă (jumătatea cea mai semnificativa a acestui registru fiind initial 0, iar in jumătatea cea mai putin semnificativa fiind incarcat deinmultitul).

Paper & pencil

$$\begin{array}{r}
 123 \times \\
 \underline{456} \\
 738 \\
 \underline{6150} \\
 49200 \\
 \hline
 56088
 \end{array}$$

50
400

INM

1011
0101
0010
0001
0000

6cmps INM (PP₀)
(PP₁)
(PP₂)
(PP₃)

DEINM 1101
INM 1011

PP₀ 1101
PP₁ 11010
PP₂ 000000
PP₃ 1101000

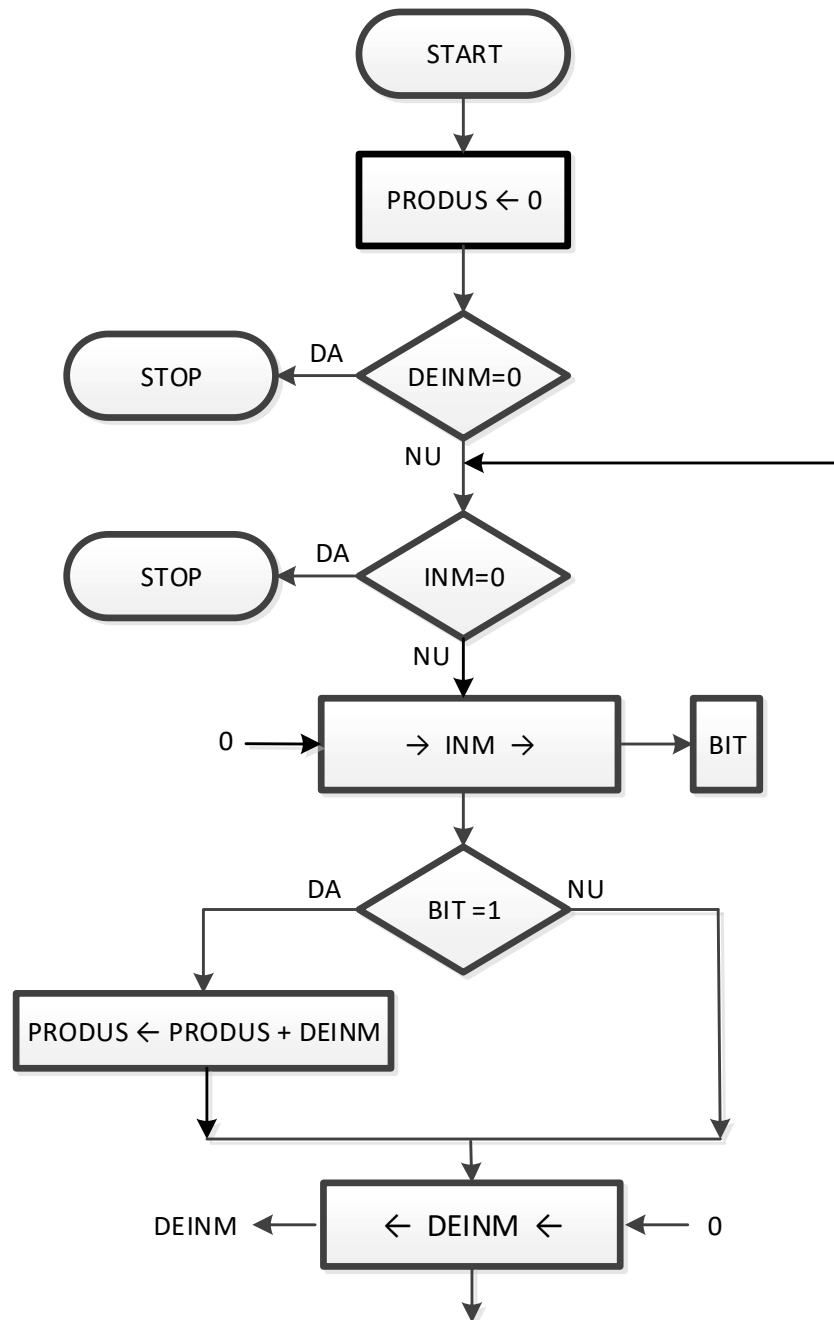
P 1000111

DEINM ← cop. depl. string
pentru produse partiale

INM → cop. depl. deapto

END 18 nov 2025

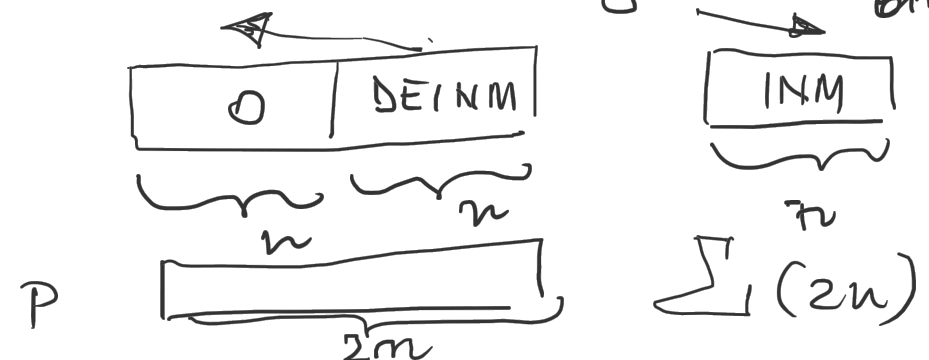
Organigrama operatiei de inmultire a 2 operanzi binari



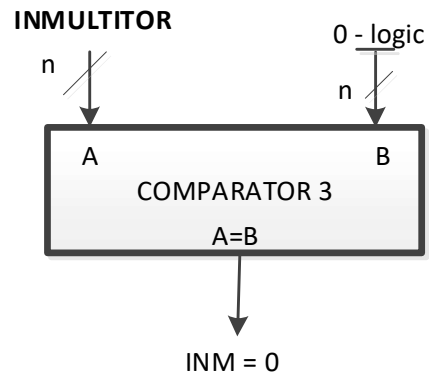
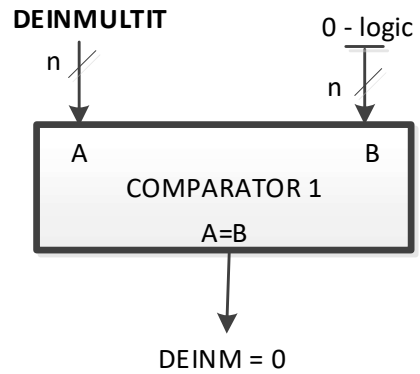
Pentru implementarea hardware este nevoie de:

- O unitate care va executa operatiile din organigrama;
- Un automat de control care va comanda unitatea de executie in conformitate cu algoritmul din organigrama.

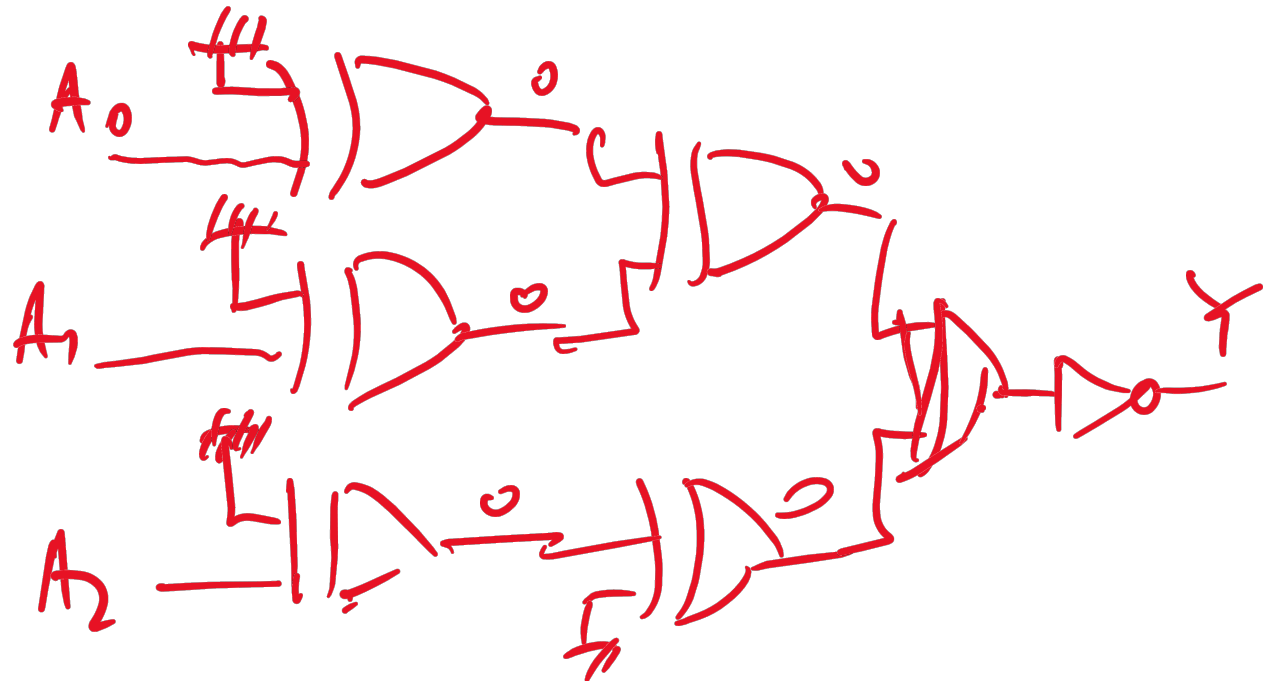
DEINM si INM - n biti
 DEINM - se pastreaza intr-un reg. dublu (2n)
 8n



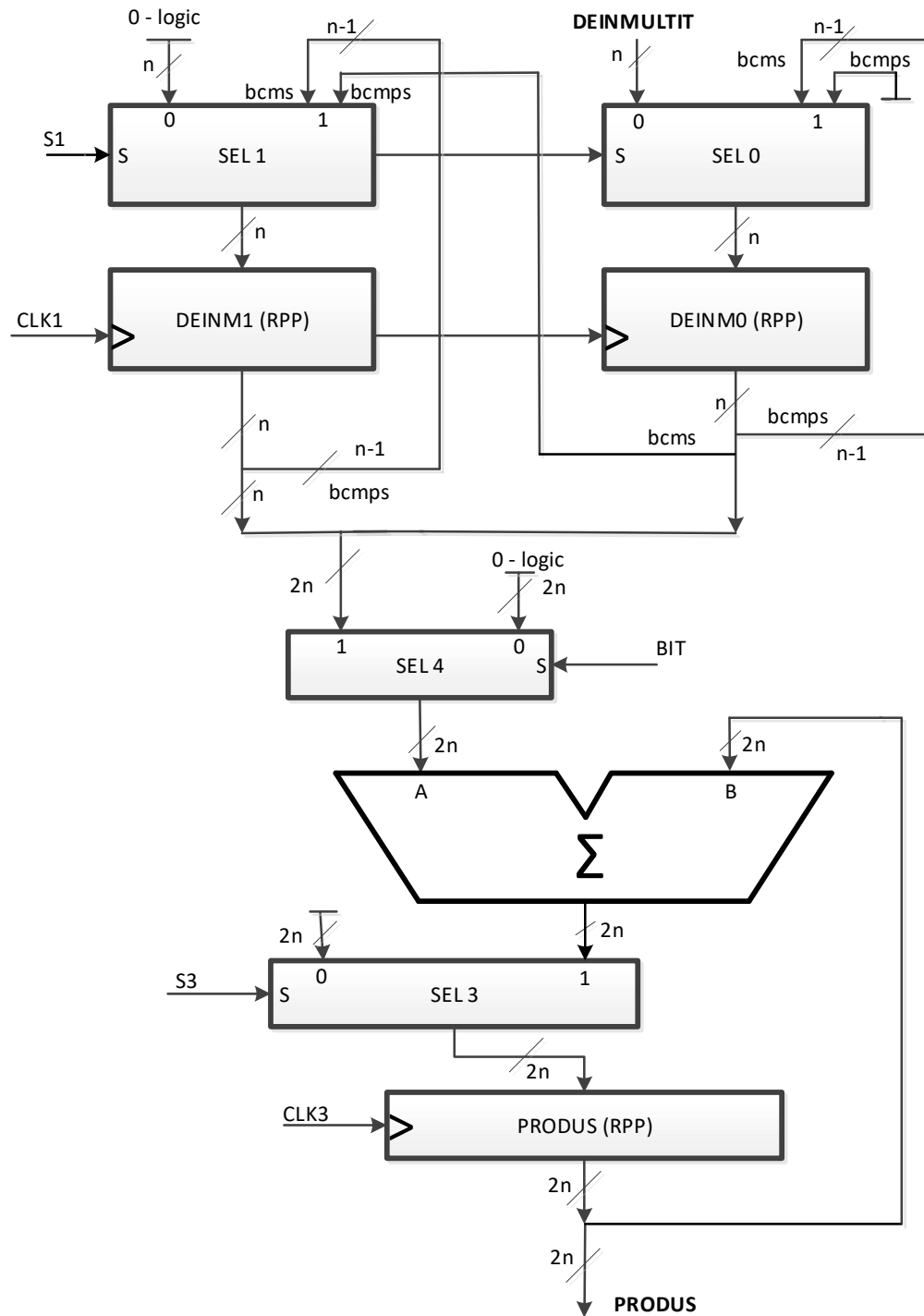
Testarea operanzilor



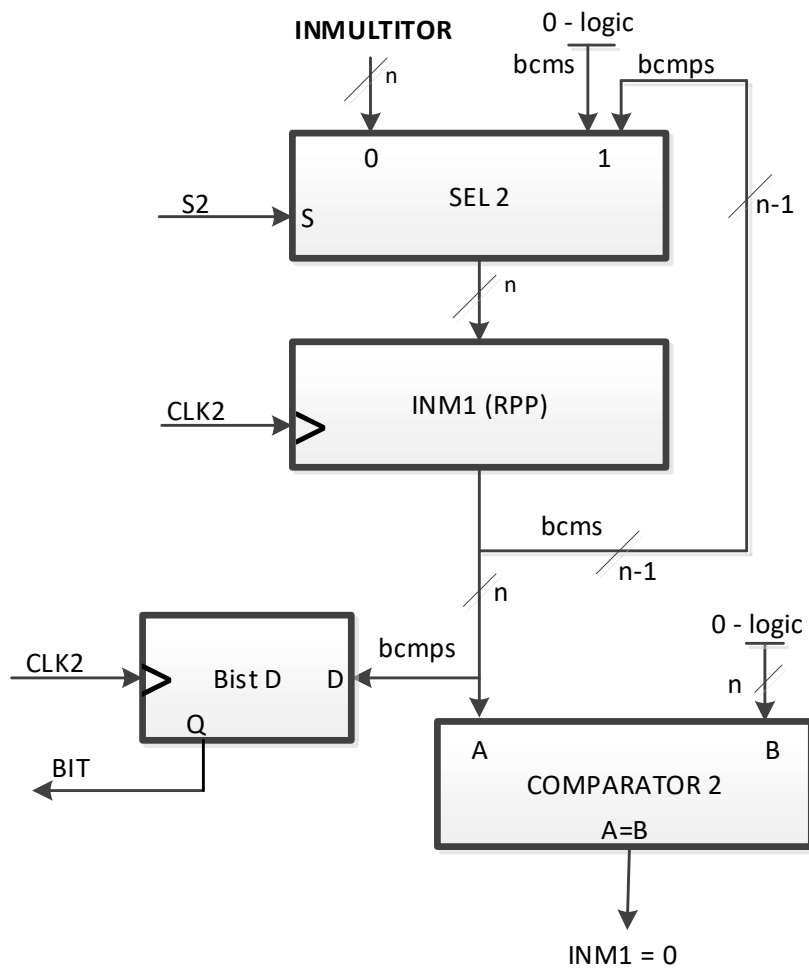
$A_3 A_2 A_1 A_0$	$Y (A-B)$
0 0 0 0	1
0 0 0 1	0
0 0 1 0	0
⋮	
1 1 1 1	1



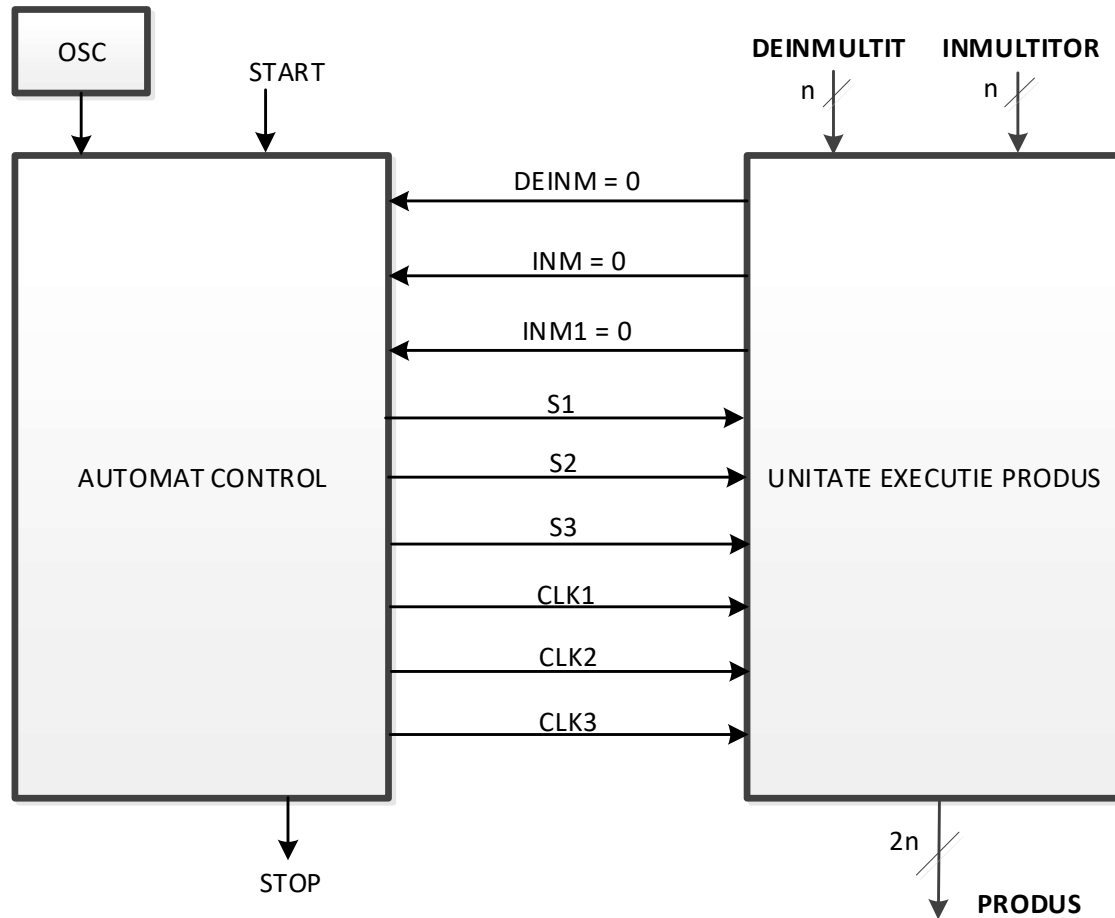
Operarea asupra de inmultitului si obtinerea produsului



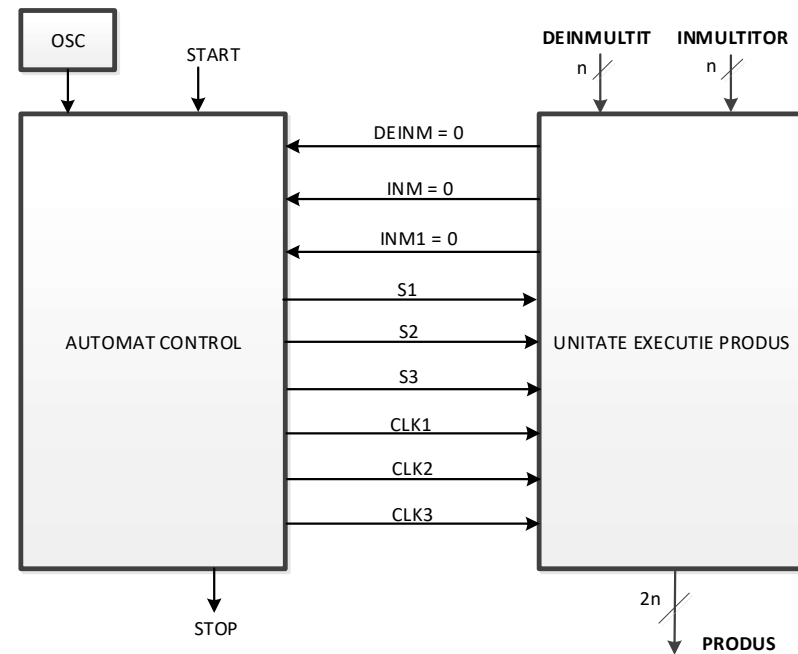
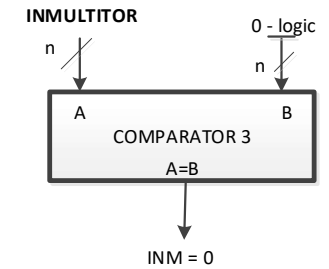
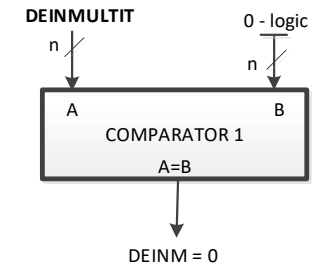
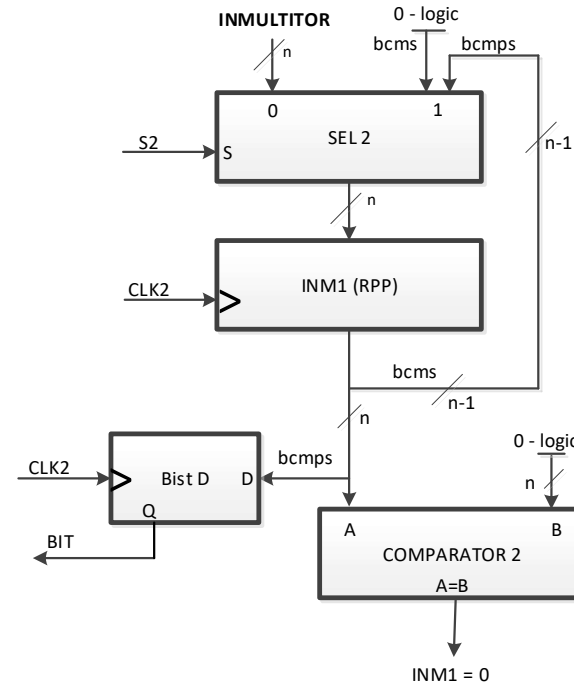
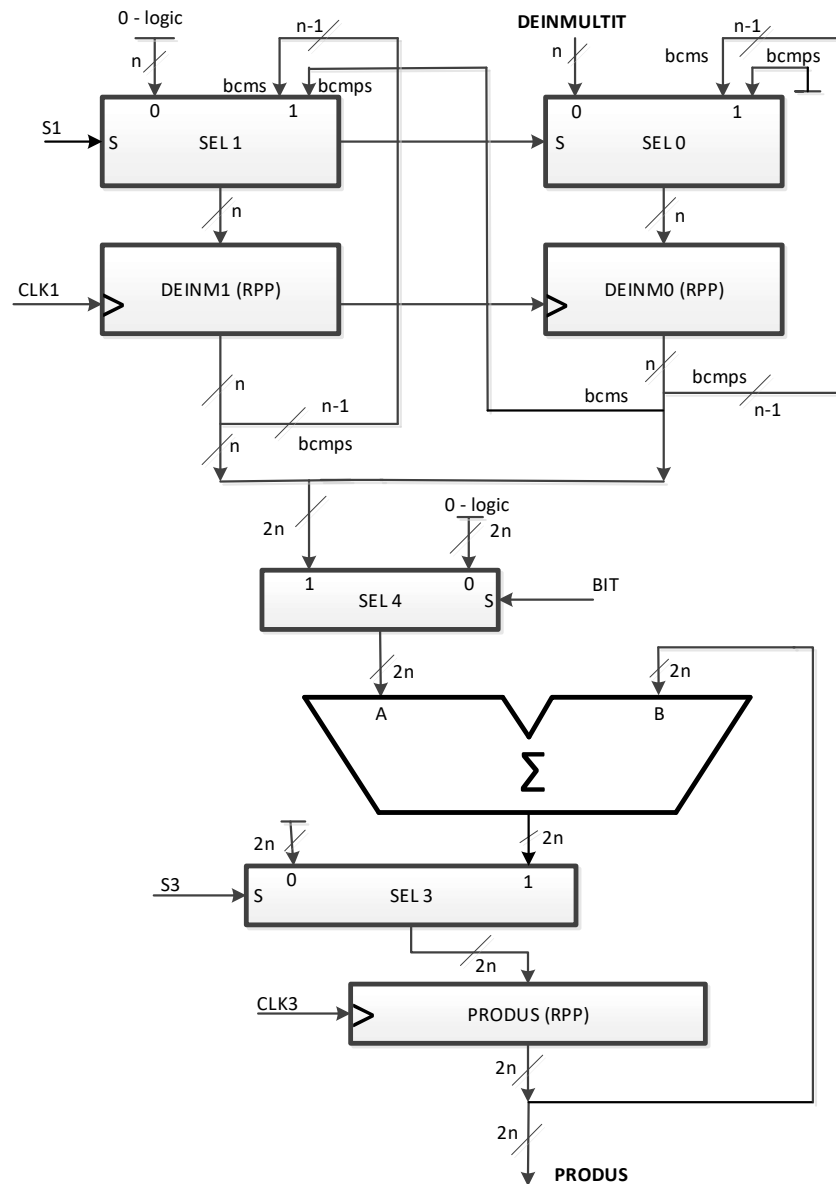
Operarea asupra inmultitorului



Schema generala a circuitului de inmultire

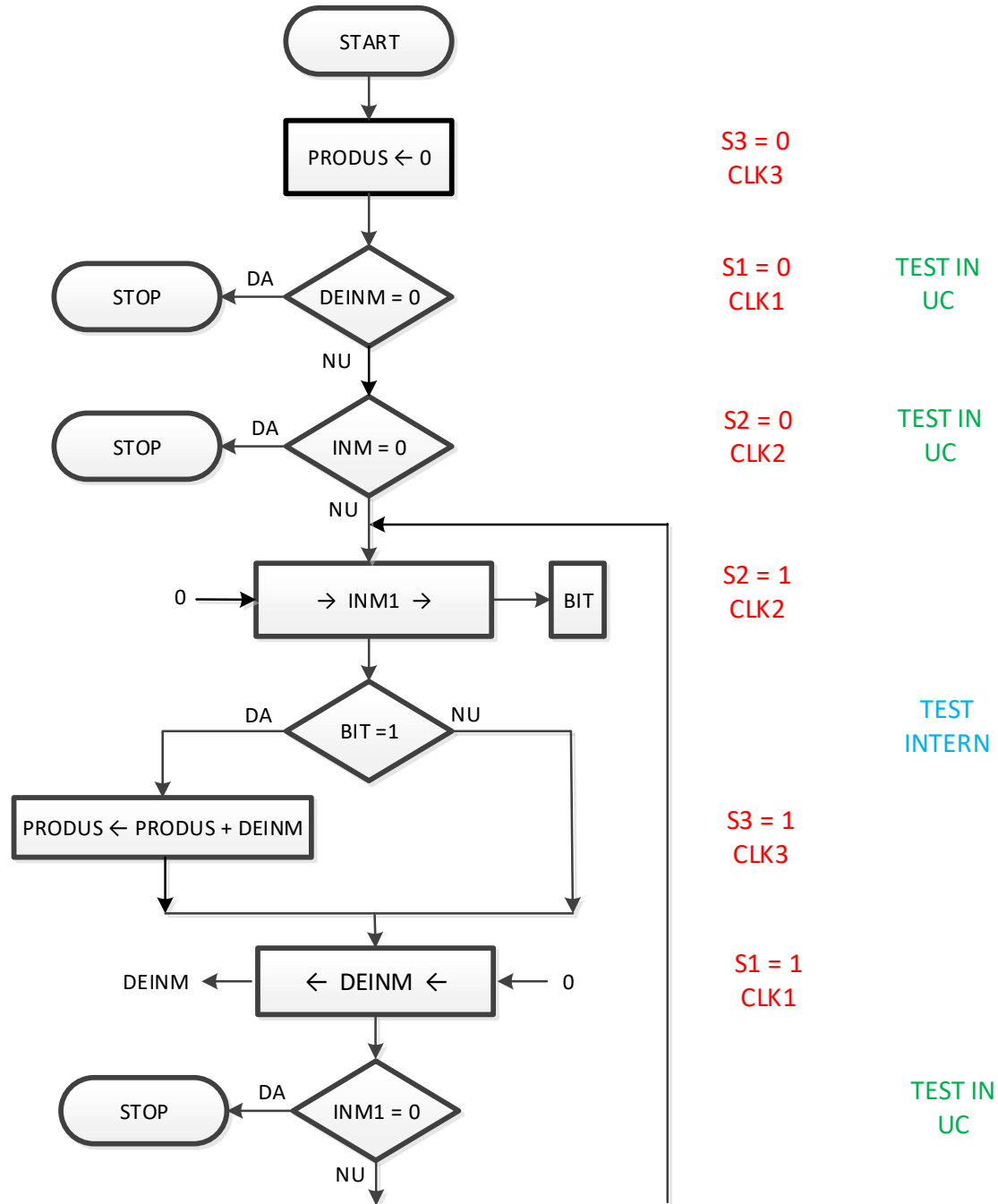


Schema hardware a circuitului de inmultire



Organigrama operatiei de inmultire a 2 operanzi binari

COMENZI
DE LA UC



Circuit optimizat pentru operatia de inmultire a 2 operanzi binari

O alta varianta pentru realizarea inmultirii a 2 operanzi se obtine prin deplasarea produsului la dreapta în locul deplasării produsului partial la stânga (ca in metoda prezentata anerior). Registrul pentru deinmultit ramane fix (nu se deplaseaza).

Avantajele acestei variante sunt următoarele:

* Este necesar un sumator de n biți, chiar dacă rezultatul va avea $2n$ biți (considerând numere fără semn).

* Deînmulțitul necesită un registru de n biți în loc de $2n$ biți.

Deoarece rezultatul are $2n$ biți, registrul pentru produs ar trebui completat cu un registru auxiliar pentru memorarea produsului complet. Acest lucru insa nu este necesar deoarece registrul pentru produs se poate plasa la stânga registrului înmulțitorului, formând cu acesta un registru combinat care se deplasează la dreapta în fiecare pas al operației. Pe masura ce se goleste inmultitorul, el este completat cu bitii cei mai puțin semnificativi din rezultat (produs).

Astfel, în final registrul pentru produs biții cei mai semnificativi ai rezultatului, iar registrul înmulțitorului păstrează biții cei mai puțin semnificativi.

Este totusi nevoie de contorizarea numarului de biti din inmultitor.

	1101 x		
	1011		

1101 x	1101		(prod.part.)
1011	-----		
-----	01101 +		(depl.dr.)
1101	1101		(deinm.)
1101	-----		
0000	100111		(prod.part.)
1101	-----		
-----	100111 +		(depl.dr.)
10001111	0000		(bit 0 in inm.)

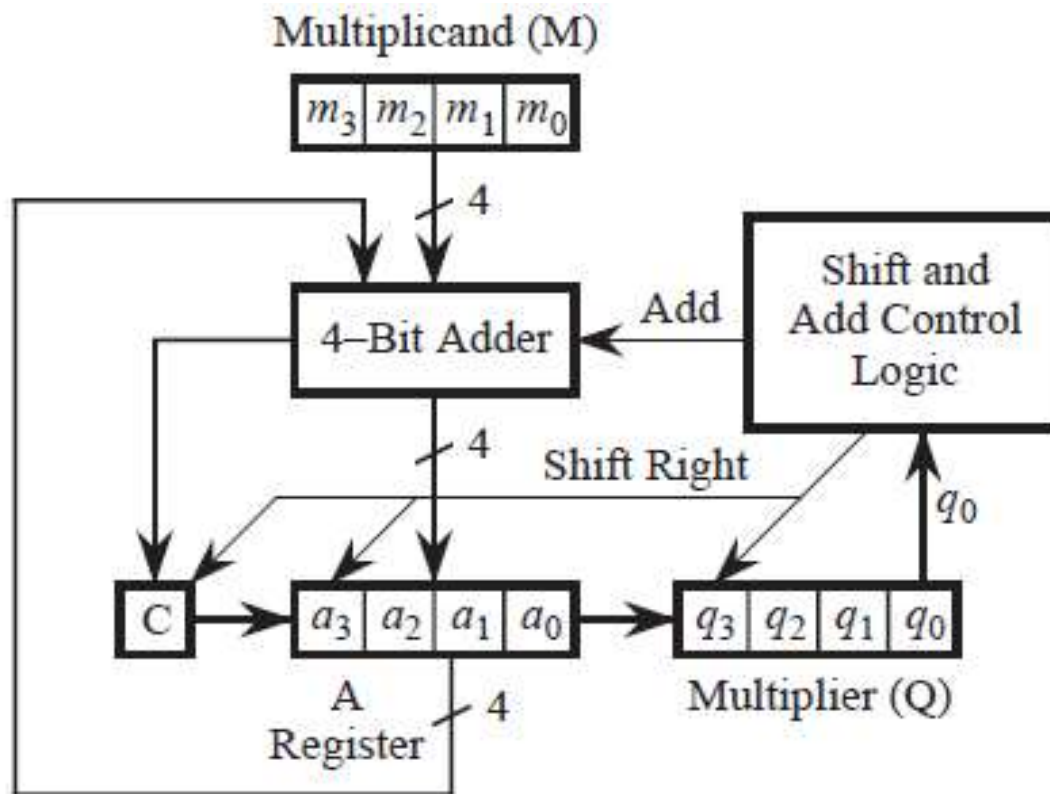
	0100111		(prod.part.)

	0100111 +		(depl.dr.)
	1101		(deinm.)

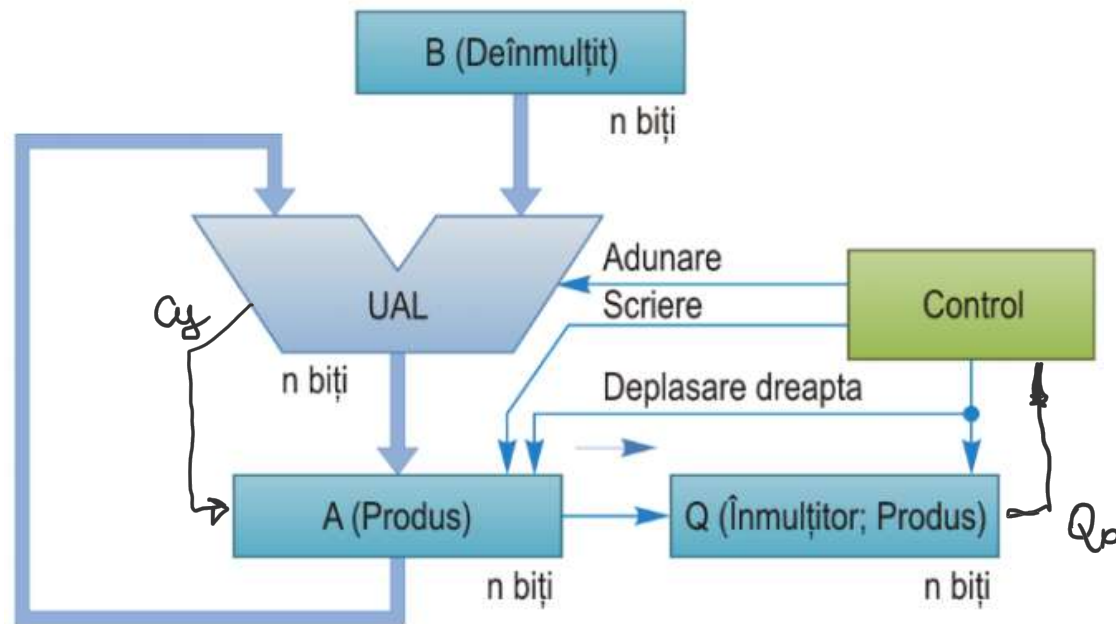
	10001111		(prod.part.)

	10001111		(depl.dr.)

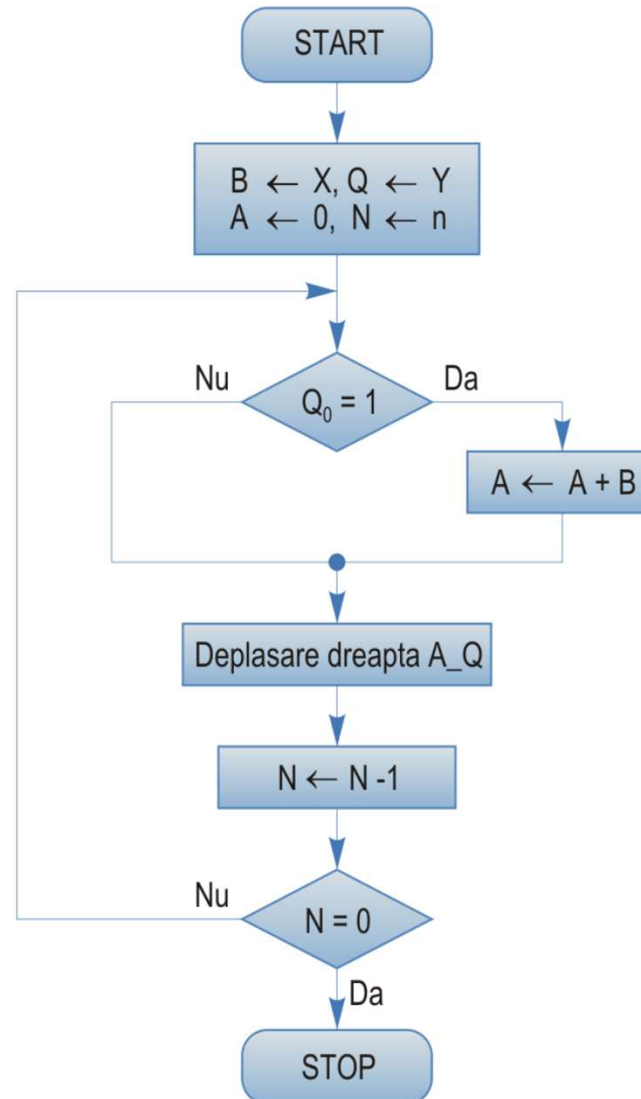
Circuit optimizat pentru operatia de inmultire a 2 operanzi binari – schema bloc



Circuit optimizat pentru operatia de inmultire a 2 operanzi binari – schema bloc



Circuit optimizat pentru operatia de inmultire a 2 operanzi binari – schema bloc



ALGORITMUL BOOTH PENTRU EFECTUAREA OPERAȚIILOR DE ÎNMULȚIRE A NUMERELOR BINARE ÎNTREGI

Pentru exemplul prezentat constatăm faptul că numărul de produse parțiale ce urmează să fie cumulate pentru obținerea produsului final este egal cu 4. Ne propunem să micșorăm acest număr de produse parțiale ce intra în calculul produsului final.

Considerăm următoarea operație de înmulțire: $101011_{(2)} \cdot 011110_{(2)}$

Pentru înmulțitor avem: $011110_{(2)} = 1 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 = 30_{(10)}$

Cunoaștem ca: $a^n - b^n = (a - b)(a^{n-1} + a^{n-2} \cdot b + \dots + a \cdot b^{n-2} + b^{n-1})$ Cu $a = 2, b = 1$ obținem

$$2^n - 1 = 2^{n-1} + 2^{n-2} + \dots + 2 + 1;$$

$$2^n - 2 = 2^{n-1} + 2^{n-2} + \dots + 2$$

Rezulta: $2^4 + 2^3 + 2^2 + 2^1 = 1 \cdot 2^5 - 1 \cdot 2^1 = 1 \cdot 2^5 + (-1) \cdot 2^1$ de unde: $011110_{(2)} = 100000_{(2)} + (-10_{(2)})$

Înmulțirea dată devine:

$$101011_{(2)} \cdot (100000_{(2)} - 10_{(2)}) = 10101100000_{(2)} - 1010110_{(2)} = 10101100000_{(2)} + (-00001010110_{(2)}) =$$

$$= 10101100000_{(2)} + 11110101010_{(2)} = 10100001010_{(2)}$$

$$(-00001010110_{(2)}) = 11110101001 + 1 = 11110101010_{(2)}$$

La ultima operație de adunare a doi operanzi în CC2 bitul de depășire este egal cu 0 și rezultatul obținut pe 11 biți este cel corect, neglijându-se bitul de transport apărut. Reprezentarea în CC2 pentru operația menționată s-a făcut pe 11 biți.

Algoritmul lui Booth propune formarea unui șir de coeficienți prin analiza biților înmulțitorului.

Termenii care apar în acest șir sunt -1, 0 sau 1, indicând și operația care se efectuează.

Șirul menționat se formează astfel:

- Înmulțitorul se analizează de la dreapta la stânga;
- Pentru primul bit 1 obținut în șirul lui Booth se alocă un coeficient -1 (produsul parțial se scade – folosind CC2);
- Dacă în continuare se regăsește un bit 1 coeficientul din șirul lui Booth este 0 (produsul parțial se deplasează stanga);
- Dacă următorul bit întâlnit este 0 coeficientul din șirul lui Booth este 1 (produsul parțial se adună);

Folosind șirul coeficienților Booth orice înmulțitor (număr binar) se poate scrie ca sume, respectiv diferențe de numere binare care în zecimal sunt puteri întregi ale lui 2.

$$\begin{array}{r}
 101011 \times \\
 011110 \\
 \hline
 101011 \\
 101011 \\
 101011 \\
 101011 \\
 \hline
 1010001010 = 50A_h
 \end{array}$$

Folosind șirul coeficienților Booth, orice înmulțitor (număr binar) se poate scrie ca sume, respectiv diferențe de numere binare care în zecimal sunt puteri întregi ale lui 2.

Ex.1 Pentru $0101111_{(2)}$ sirul Booth este $1 -1 \ 1 \ 0 \ 0 \ 0 \ -1$ rezultand $101111_{(2)} = 1000000_{(2)} - 100000_{(2)} + 10000_{(2)} - 1$

Se constata ca prin algoritmul Booth obtinem 4 produse partiale, fata de 5 produse partiale in algoritmul clasic.

Ex.2 Pentru $011011_{(2)}$ sirul Booth este $1 \ 0 \ -1 \ 1 \ 0 \ -1$ $11011_{(2)} = 100000_{(2)} - 1000_{(2)} + 100_{(2)} - 1$

Se constata ca prin algoritmul Booth obtinem 4 produse partiale, similar cu 4 produse partiale care apar in algoritmul clasic.

Ex.3 Pentru $010101_{(2)}$ sirul Booth este $1 \ -1 \ 1 \ -1 \ 1 \ -1$ $10101_{(2)} = 100000_{(2)} - 10000_{(2)} + 1000_{(2)} - 100_{(2)} + 10_{(2)} - 1$

Se constata ca prin algoritmul Booth obtinem 6 produse partiale, fata de 3 produse partiale care apar in algoritmul clasic.

În concluzie, această primă formă a algoritmului lui Booth este eficientă atunci când șirul de biți 1 logic aflați în înmulțitor este mai mare ca 2.

Exemplu $21 \times 14 = 294$

0 1 0 1 0 1	×	Deinm
0 0 1 1 1 0		Inm

0 +1 0 0 -1 0		Sir Booth Inm

(- 0101010)	+	Prod.part.1
(0101010000)		Prod.part.2

0 1111010001		Transport
1111010101	+	CC2 PP1
1		
0101010000		PP2

0100100110		Produs

0 = 0		0 = 0

ALGORITMUL BOOTH MODIFICAT

Acest algoritm oferă soluții pentru situația în care șirul de biți 1 logic aflați în înmulțitor nu este mare, inclusiv pentru situația biților 1 solitari. În acest algoritm modificat se propune un nou șir de coeficienți derivați din algoritmul lui Booth normal. Aceștia apar din analiza fiecărui grup de câte 2 termeni din șirul Booth normal, plecând din dreapta către stânga. În noul șir termenii care pot să apară sunt ± 2 , ± 1 și 0.

Sucesiune biți în nr binar	Coeficienți în Booth normal	Coeficient în Booth modificat
0 0 0	0 0	0
1 1 1		
0 1 1	1 0	+2
0 0 1	0 1	+1
1 0 0	-1 0	-2
0 1 0	1 -1	+1
1 1 0	0 -1	-1
1 0 1	1 -1	+1

Exemplu $14 \times 21 = 294$

0 0 1 1 1 0	×	Deinm
0 1 0 1 0 1		Inm
1 -1 1 -1 1 -1		Sir Booth Inm
↓ ↓ ↓		
+1 +1 +1		Sir Booth mod.
2^4 2^2 2^0		Ponderi coef.
0 0 1 1 1 0		PP1
0 0 1 1 1 0 0 0		PP2
0 0 1 1 1 0 0 0 0 0		PP3
0 0 0 1 1 1 1 0 0 0		Transport
0 1 0 0 1 0 0 1 1 0		Produs

END 02 dec 2025